

UC Santa Cruz

UC Santa Cruz Electronic Theses and Dissertations

Title

Testing Memory Models of Heterogeneous CPU-GPU Systems

Permalink

<https://escholarship.org/uc/item/5hm7q1jv>

Author

Srivastava, Sanya

Publication Date

2024

Peer reviewed|Thesis/dissertation

UNIVERSITY OF CALIFORNIA

SANTA CRUZ

**TESTING MEMORY MODELS OF HETEROGENEOUS CPU-GPU
SYSTEMS**

A thesis submitted in partial satisfaction
of the requirements for the degree of

MASTER OF SCIENCE

in

COMPUTER SCIENCE AND ENGINEERING

by

Sanya Srivastava

June 2024

The Thesis of Sanya Srivastava
is approved:

Professor Tyler Sorensen, Chair

Professor Andrew Quinn

Professor Peter Alvaro

Peter Biehl
Vice Provost and Dean of Graduate Studies

Table of Contents

List of Figures	v
List of Tables	viii
Abstract	x
Acknowledgments	xi
1 Introduction	1
1.1 Motivation	1
1.2 Contributions and Thesis Outline	2
2 Background	5
2.1 Memory Consistency Models and Reordering	5
2.1.1 Weak Memory Behavior	7
2.1.2 Targeted Litmus Tests	8
2.2 Formalizing Executions	12
2.3 Empirical Testing	14
2.3.1 Single Instance Testing	15
2.3.2 Perpetual Instance Testing	16
2.4 Heterogeneous Testing	18
2.5 Important Symbols and Terminologies	18
3 Compiler Design	21
3.1 Algorithm at a Glance	22
3.2 Compiler Input	26
3.2.1 Cyclic Execution Graph	27
3.2.2 Fine-Grained Configurations	28
3.2.3 Template Files and Output Files	31
3.3 Observer Threads	33

3.4	Choosing Values for the Write Nodes	35
3.5	Generating Trace Analysis Heuristics	38
3.5.1	Single Instance Tests	38
3.5.2	Perpetual Instance Tests	42
3.6	Optimizations Injected to Observe More Weak Behavior	64
4	Experiments and Results	68
4.1	Devices Tested	68
4.2	Experiments Performed	70
4.3	Single Instance Test Results	76
4.3.1	Analysis	79
4.4	Perpetual Instance Test Results	84
4.4.1	Analysis	88
5	Future Work and Conclusion	104
5.1	Future Work	104
5.2	Conclusion	108
A	Raw Data	109
	Bibliography	120

List of Figures

2.1	Example demonstrating sequential (execution 1 and 2), interleaved (execution 3), and weak behavior execution (execution 4) for the Message Passing test.	9
2.2	Message Passing litmus test.	9
2.3	2 + 2 Write litmus test.	10
2.4	Store litmus test.	11
2.5	Read litmus test.	11
2.6	Store Buffer litmus test.	12
2.7	Load Buffer litmus test.	12
2.8	Deconstruction of an <i>fr</i> edge into a <i>ws</i> edge and <i>rf</i> edge.	13
2.9	Graph representation of weak memory behavior execution for the Store Litmus test	14
2.10	Litmus testing on heterogeneous CPU-GPU systems.	19
3.1	Flowchart representing different tests that can be produced by our compiler. At depth 2, the nodes show the type of tests we can generate. Depth 3 nodes show the backends we have for each of these test types, and depth 4 nodes show the different empirical testing methodologies we can use to generate these tests.	22
3.2	Coarse-grained view of the compiler algorithm.	23
3.3	Barrier synchronization example using three test threads.	26

3.4	Example demonstrating observer threads. The yellow bar on top of each thread represents the barrier synchronization. X and Y are locations/addresses on shared memory.	35
3.5	Graph traversal on the 2 + 2 Write litmus test execution graph to determine conditions for weak memory behavior.	42
3.6	Graph traversal on the Store litmus test execution graph to determine conditions for weak memory behavior.	43
3.7	Example demonstrating how <i>fr</i> edges are handled by the compiler. . . .	51
3.8	2+2 Write Litmus test graph.	55
3.9	Example demonstrating the concept of synchrony point.	59
3.10	Store Buffer litmus test graph.	63
3.11	Message Passing litmus test graph.	63
3.12	Load Buffer litmus test graph.	64
3.13	Store litmus test graph.	64
4.1	Post processing runtime (seconds) for Store test runs with 1000 iterations.	76
4.2	Post processing runtime (seconds) for Store test runs with 3000 iterations.	77
4.3	Weak memory behavior percentage observed with different optimizations for 2 + 2 Write single instance test configurations.	85
4.4	Weak memory behavior percentage observed with different optimizations for Load Buffer single instance test configurations.	86
4.5	Weak memory behavior percentage observed with different optimizations for Message Passing single instance test configurations.	87
4.6	Weak memory behavior percentage observed with different optimizations for Store Buffer single instance test configurations.	88
4.7	Weak memory behavior percentage observed with different optimizations for Store single instance test configurations.	89
4.8	Weak memory behavior percentage observed with different optimizations for Read single instance test configurations.	90

4.9	Test direction impact on weak behavior percentage for 2 + 2 Write single instance test configurations.	91
4.10	Test direction impact on weak behavior percentage for Load Buffer single instance test configurations.	92
4.11	Test direction impact on weak behavior percentage for Message Passing single instance test configurations.	93
4.12	Test direction impact on weak behavior percentage for Store Buffer single instance test configurations.	94
4.13	Test direction impact on weak behavior percentage for Store single instance test configurations.	95
4.14	Test direction impact on weak behavior percentage for Read single instance test configurations.	96
4.15	Change in throughput with increasing window size.	97
4.16	Throughput for different perpetual instance test configurations of 2+2 Write litmus test with and without stress.	98
4.17	Impact of window size on weak memory behavior throughput for different perpetual instance test configurations of 2+2 Write litmus test with memory stress enabled.	99
4.18	Weak memory behavior throughput for different test configurations of the MP litmus test using the perpetual litmus testing approach.	100
4.19	Weak memory behavior throughput for different test configurations of the SB litmus test using the perpetual litmus testing approach.	100
4.20	Weak memory behavior throughput for different test configurations of the LB litmus test using the perpetual litmus testing approach.	101
5.1	Converting the 2+2 Write litmus test into a Message Passing litmus test using observer threads.	106

List of Tables

4.1	Experiments performed with single-instance approach. In the table, the GPU fence ‘blockfence’ refers to the fence <code>__threadfence_block()</code> . The fence ‘systemfence’ refers to the fence <code>__threadfence_system()</code> , and the fence ‘threadfence’ refers to the fence <code>__threadfence()</code>	71
4.2	Experiments performed with perpetual-instance approach. In the table, the GPU fence ‘blockfence’ refers to the fence <code>__threadfence_block()</code> . The fence ‘systemfence’ refers to the fence <code>__threadfence_system()</code> , and the fence ‘threadfence’ refers to the fence <code>__threadfence()</code>	71
4.3	Store litmus test perpetual instance significant results.	75
4.4	Average weak behaviors seen using different fence-direction configurations for the litmus tests	81
A.1	Message Passing litmus test - single instance results.	110
A.2	Store Buffer litmus test - single instance results.	111
A.3	Load Buffer litmus test - single instance results.	112
A.4	Read litmus test - single instance results.	113
A.5	Store litmus test - single instance results.	114
A.6	2 + 2 Write litmus test - single instance results.	115
A.7	2 + 2 write litmus test - perpetual instance results.	116
A.8	Load Buffer litmus test - perpetual instance results.	117
A.9	Store Buffer litmus test - perpetual instance results.	118

A.10 Message Passing litmus test - perpetual instance results.	119
--	-----

Abstract

Testing Memory Models of Heterogeneous CPU-GPU Systems

by

Sanya Srivastava

To efficiently exploit the growing heterogeneity in systems, we need to know their software-hardware agreement for instruction reordering. Without the knowledge of these underlying contracts and how shared memory behaves on heterogeneous systems, concurrent code cannot be written safely, as unexpected reorderings by the hardware can cause data races that might not exist in an architecture with a sequentially consistent memory model. These agreements become especially complex in heterogeneous platforms because different systems can allow different reorderings, making it crucial to understand the behavior of shared memory. This work aims to explore the memory consistency model (MCM) of integrated CPU-GPU systems. We propose a compiler to generate memory consistency tests for empirically testing the MCM of these heterogeneous systems. We show that all four memory reorderings can be observed on the NVIDIA Jetson Orin Nano. However, the reorderings can be eliminated by using fences of appropriate strengths.

Acknowledgments

I would like to thank Professor Tyler Sorensen, my advisor, who gave me the excellent opportunity to work on this project and constantly supported me throughout this process. I am very grateful to him for providing rigorous feedback during the weekly one-on-one meetings and teaching me how to develop ideas and write a thesis. His mentorship and guidance have been invaluable to me.

I am incredibly grateful to Professor Peter Alvaro and Professor Andrew Quinn for guiding me through the thesis and providing feedback that was instrumental in refining my work.

Additionally, this project was supported in part by the NSF, Award CCF 2239400, which helped make this research possible.

Lastly, I would like to thank my friends and family. Without their love and support, this accomplishment would not have been possible.

Chapter 1

Introduction

1.1 Motivation

Recent architectural trends show that systems are becoming more heterogeneous due to their powerful computing capabilities [22, 6]. However, if we want to enable fine-grained communication between these integrated devices to exploit the computing power of the heterogeneous system, we need to know the memory model that combines them. This is because designing safe concurrent programs without the knowledge of the reorderings and behaviors allowed and how to eliminate them using fences is almost impossible. Moreover, designing concurrency primitives, such as mutexes, requires a sequentially consistent memory model, and different architectures, especially heterogeneous architectures, are not inherently sequentially consistent. These systems typically have a weaker memory model that permits certain reorderings. Understanding and eliminating these reorderings is crucial for moving toward a more sequential model that fits the purpose of our program.

Academics have only recently begun to investigate the memory models of hetero-

geneous systems [5, 9]. There are tools that test memory consistency models of CPUs, such as litmus7 [8] and PerpLE [15], or GPUs only [4, 12, 13], but we don't have parallel tools to test the memory consistency models of existing heterogeneous systems, such as integrated CPU-GPU systems that are widely used, especially in Android and Apple devices. For example, the Snapdragon 888, which has a Qualcomm Adreno 660 GPU and a Qualcomm Kryo CPU [20], is used in Samsung devices; the Kirin 9000, incorporating an Arm Cortex-A77 CPU and a Mali-G78 GPU [7], is used in Huawei devices, and the heterogeneous Apple bionic chips are used in recent iPhone models.

The primary motivation for this work is to develop a memory consistency testing framework and methodology that is especially well suited for heterogeneous systems. Our goal is to match or exceed the performance of the existing GPU and CPU-only testing tools like litmus7 [8]. Using our framework, we want to rigorously test these heterogeneous CPU-GPU systems and formulate a concrete analysis of the reorderings that exist and how to mitigate them.

1.2 Contributions and Thesis Outline

Contributions:

- We developed a compiler to generate memory consistency tests. To our knowledge, our compiler is the only tool that accepts execution cycles that encode weak memory behaviors instead of program pseudo-code or assembly instructions to generate the corresponding test files. Our compiler can be used to generate two kinds of tests: single-instance and perpetual-instance [15]. We explain the differences between these two approaches in depth in Section 2.3. Using our compiler

framework, we can not only test for memory consistency but also for cache coherence by using appropriate tests.

- We proposed a methodology to extend the applicability of the original perpetual instance testing approach introduced by Melissaris et al. in their paper ‘PerpLE: Improving the Speed and Effectiveness of Memory Consistency Testing’ [15]. We found that the original perpetual instance testing approach was only applicable/suitable for programs with at least 2 read instructions. However, we extend the approach so we can test different kinds of litmus tests that have 1 or even 0 read instructions.
- We developed a CUDA backend, which allows us to test devices that have NVIDIA GPUs, and a Vulkan backend, which allows us to test devices that have Intel, AMD, Qualcomm, Imagination, and ARM GPUs.
- We generated CUDA heterogeneous (CPU-GPU) single instance and perpetual instance tests for different litmus tests detailed in Section 2.1.2. Through these tests, we tested shared memory on the 8GB NVIDIA Jetson Orin Nano. We found that we can see all four possible reorderings (see Sec. 2.1) on the device. Perpetual instance tests gave us a considerably larger weak behavior throughput (number of weak memory behaviors observed per second) than single instance tests. However, we were able to eliminate all reorderings by using a `__threadfence_systemfence()` between instructions in the GPU thread.

Thesis Roadmap: Chapter 2 will discuss the background of the work, including some core concepts, such as weak memory behavior, memory reorderings, and litmus tests that will be utilized throughout the thesis. Chapter 3 will discuss the design and im-

plementation of the compiler, one of the primary contributions of this thesis, that we have built for testing. This chapter will also cover the concepts about perpetual instance testing that we had to implement to extend its functionality and the optimizations we added to reduce the time complexity of tests and increase the weak memory behavior throughput. Chapter 4 will discuss our testing setup and analysis of results, contrasting the differences between single-instance results and perpetual-instance results. Finally, Chapter 5 will go over the conclusion and future works that we plan on doing as a continuation of this project.

Chapter 2

Background

2.1 Memory Consistency Models and Reordering

The memory consistency model of a system can be thought of as an agreement or a contract that defines the order in which instructions can be processed. In formal terms, a memory consistency model "specifies what values dynamic loads may return and what possible final states of the memory are" [16]. When the hardware is allowed to reorder a combination of load and store instructions, then the memory consistency model is considered weak or relaxed. We have a sequentially consistent model when the hardware does not perform any reorderings in memory instructions. Leslie Lamport provides a formal definition of a sequential consistent model, stating that a model is sequentially consistent if,

"the result of any execution is the same as if the operations of all the processors were executed in some sequential order, and the operations of each individual processor appear in this sequence in the order specified by its program" [11]

Hence, a sequentially consistent model is one in which the order of instructions pro-

vided by the software remains unchanged [1]. In this work, we restrict the memory models to global happens-before models [2]. These models only allow for reorderings in program order, which include:

- **Store $\rightarrow$ Store:** This memory reordering happens between 2 stores that write to different memory addresses. Hence, the write made by the second store can propagate before the first store operation's write. Cores with non-sequential store buffers allow for this reordering [16].
- **Store $\rightarrow$ Load:** This reordering means that if a store memory instruction is followed by a load instruction (both operating on different memory addresses), then they are allowed to get reordered. The TSO (Total-Store order) memory model allows for this reordering [16].
- **Load $\rightarrow$ Store:** This reordering allows for a load instruction reading from some memory address X followed by a store instruction that is writing to a different memory address ($\neq X$) to get reordered.
- **Load $\rightarrow$ Load:** This reordering allows for two load instructions reading from different memory addresses to get reordered.

For example, consider the program shown in Figure 2.6. Now, if all instructions are processed sequentially, then we should never be able to see an outcome where $r0$ is 0, and $r1$ is 0 after the program is completed. However, this outcome is valid on the TSO memory model. This is because TSO allows for Store $\rightarrow$ Load reordering. However, if we take the program shown in Figure 2.3, and execute it on the TSO model, then we will never be able to see an outcome where $X = 1$ and $Y = 1$ after the program is

finished because there is no Store $\rightarrow$ Store reordering on TSO. Both these behaviors of TSO can be reasoned by the local first-in-first-out write buffers [14, 16].

For disallowing these reorderings, we can insert fences in between instructions. Different memory models specify fences of varying strengths. For example, if the store buffer test shown in Figure 2.6 is executed on an x86 CPU that has the TSO memory model, and a fence is added between the Store and Load instructions, then we should not be able to see the outcome observed earlier. This is because the fence will force the write buffer to flush all the write instructions, causing the updates to propagate before the load instruction is executed. This ensures that the load instruction following the fence reads the updated value.

2.1.1 Weak Memory Behavior

A weak memory behavior happens when we get a non-sequential outcome for a program. Every instruction in a program gives rise to events, and we can think about these events as Lego bricks. We can arrange/stack these Lego bricks from different threads while respecting the program order and construct a global timeline of events to get the final outcome of a program. When we have a program execution outcome that cannot be obtained by stacking these Lego bricks, then we call that outcome/result of the execution a weak memory behavior.

Consider the program shown in Figure 2.2. Figure 2.1 shows the possible outcomes we can get by arranging the Lego bricks. The blue bricks represent thread 0 (T_0) events, and the red bricks represent thread 1 (T_1) events. In execution 1, T_0 gets executed before T_1 , and both the read events read the values that were stored by the write events of T_0 . In execution 2, T_1 gets executed before T_0 , and the read events

read the values the memory addresses were initialized to (0s). Execution 3 shows an interleaved behavior in which the first event from $T0$ gets executed, then the first event from $T1$, followed by the second event of $T0$. The second event of $T1$ is executed in the end. In this case, $r0$ will have the un-updated value (0), and $r1$ will have the updated value (1). Then, we have a 4th outcome in which $r0$ has 1, and $r1$ has 0. Arranging the Lego bricks for this outcome is not possible. For $\text{Read}[Y]$ to read the value 1, it will have to come after the second instruction of $T0$ in the global timeline. Placing it after the last instruction of $T0$ means the $\text{Read}[X]$ will also have to read the value 1, because the first instruction of thread 0 will have to get executed before the second. Hence, there is no possible way we can arrange the blocks for this outcome on the global timeline, and so this behavior is a weak memory behavior. For this outcome to be possible, either $\text{Read}[X]$ will have to be reordered with $\text{Read}[Y]$, or the write instructions will have to be reordered (shown in red dotted arrows).

2.1.2 Targeted Litmus Tests

For testing such reorderings and weak memory behaviors, we can use litmus tests [14], which are small programs that encode a weak memory execution. In this thesis, we will focus on two-threaded litmus tests with two instructions each, specifically on 6 different litmus tests that have been described below:

Message Passing: The Message Passing litmus test was discussed in Section 2.1.1 and has a consumer-producer pattern. In this test, thread 0 has two store instructions writing to two different memory addresses, and thread 1 has two read instructions reading from them in the reverse order. The test is shown in Figure 2.2. A weak memory behavior will happen when the first load instruction reads the updated value and the

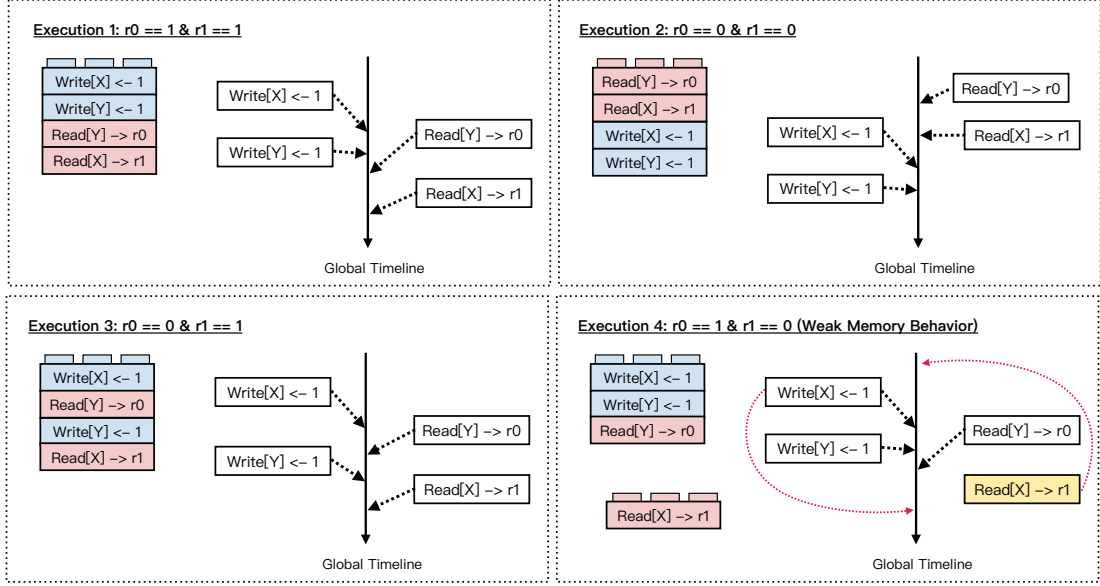


Figure 2.1: Example demonstrating sequential (execution 1 and 2), interleaved (execution 3), and weak behavior execution (execution 4) for the Message Passing test.

second load instruction reads in un-updated value.

Initialize: $X = 0, Y = 0$	
<u>Thread 0</u>	<u>Thread 1</u>
a. Store(X) $\leftarrow$ 1;	c. Load(Y) $\rightarrow$ r0;
b. Store(Y) $\leftarrow$ 1;	d. Load(X) $\rightarrow$ r1;
Weak Behavior: $r0 = 1 \wedge r1 = 0$	

Figure 2.2: Message Passing litmus test.

2 + 2 Write: This litmus test has two store instructions in thread 0 and two store instructions in thread 1 (shown in Figure 2.3). We have a weak memory behavior when the final value in the memory locations X and Y is 1. Because instructions c and d in this test write the value 2 to the memory locations, reading the value 1 after the test is

done indicates a Store $\rightarrow$ Store memory reordering.

Initialize: $X = 0, Y = 0$	
<u>Thread 0</u>	<u>Thread 1</u>
a. Store(X) $\leftarrow$ 1;	c. Store(Y) $\leftarrow$ 1;
b. Store(Y) $\leftarrow$ 2;	d. Store(X) $\leftarrow$ 2;
Weak Behavior: $X = 1 \wedge Y = 1$	

Figure 2.3: 2 + 2 Write litmus test.

Store and Read: The Store and Read litmus tests are similar. Both have two store instructions in thread 0 and then a load and store instruction in thread 1. The difference is that the Store test has a Load $\rightarrow$ Store sequence in thread 1, and the Read test has a Store $\rightarrow$ Load sequence in thread 1. We see a weak behavior in the Store test when the final value in the memory location X is 2 (comes from the first store instruction in thread 0), and the value read by the load instruction in thread 1 is 1 (comes from the second instruction in thread 0). If $r0$ should read 0, then instruction c should come after instruction b , which means X 's final value cannot be 2 without reordering in these memory instructions. Similarly, for the Read litmus test, a weak memory behavior is when we have the value 2 in Y (coming from the first instruction in thread 1), and the load instruction reads a 0. For this behavior to be possible, instruction d will have to come before any of the stores, which means thread 1 should get executed first. However, if instruction c gets executed first, then the outcome $Y = 2$ will not be possible in the end. These tests are shown in Figures 2.4 and 2.5.

Store Buffer and Load Buffer: The Store Buffer and Load Buffer tests are similar. In the Store Buffer test, we have a Store $\rightarrow$ Load sequence in thread 0 and thread 1. In

Initialize: $X = 0, Y = 0$	
<u>Thread 0</u>	<u>Thread 1</u>
a. Store(X) $\leftarrow$ 2;	c. Load(Y) $\rightarrow$ r0;
b. Store(Y) $\leftarrow$ 1;	d. Store(X) $\leftarrow$ 1;
Weak Behavior: $X = 2 \wedge r0 = 1$	

Figure 2.4: Store litmus test.

Initialize: $X = 0, Y = 0$	
<u>Thread 0</u>	<u>Thread 1</u>
a. Store(X) $\leftarrow$ 1;	c. Store(Y) $\leftarrow$ 2;
b. Store(Y) $\leftarrow$ 1;	d. Load(X) $\rightarrow$ r0;
Weak Behavior: $r0 = 0 \wedge Y = 2$	

Figure 2.5: Read litmus test.

the Load Buffer test, the instruction order in the threads is flipped, and we have a Load $\rightarrow$ Store sequence instead. In the Store Buffer test, we have a weak memory behavior when both the read instructions read the initialization value (0). This is a weak behavior because for $r0$ and $r1$ to read 0, these instructions will have to happen before any stores, which is not possible without a Store $\rightarrow$ Load reordering. Similarly, we have a weak memory behavior for the Load Buffer test when both the load instructions read the values written by the stores, but both the load instructions are sequenced before any store instructions in the threads. A weak memory behavior in the Load Buffer litmus test corresponds to a Load $\rightarrow$ Store reordering. These tests are shown in Figures 2.6 and 2.7.

There are other litmus tests, such as IRIW, RWC, and IRRWIW [18, 19], that have more than 2 threads, but they are not the focus of the thesis and our experiments. However, all such tests can be generated from our compiler, which is described in the next

chapter.

Initialize: $X = 0, Y = 0$	
<u>Thread 0</u>	<u>Thread 1</u>
a. Store(X) $\leftarrow 1$;	c. Store(Y) $\leftarrow 1$;
b. Load(Y) $\rightarrow r0$;	d. Load(X) $\leftarrow r1$;
Weak Behavior: $r0 = 0 \wedge r1 = 0$	

Figure 2.6: Store Buffer litmus test.

Initialize: $X = 0, Y = 0$	
<u>Thread 0</u>	<u>Thread 1</u>
a. Load(X) $\rightarrow r0$;	c. Load(Y) $\rightarrow r1$;
b. Store(Y) $\leftarrow 1$;	d. Store(X) $\leftarrow 1$;
Weak Behavior: $r0 = 1 \wedge r1 = 1$	

Figure 2.7: Load Buffer litmus test.

2.2 Formalizing Executions

In the previous sections, we used informal terminology to explain where the value read by a load instruction comes from, or how should the order of instructions on the same thread be preserved to show how different instructions/events were connected in the timeline. Instead of this informal notation, we can formalize the test executions by representing them as graphs. The nodes in the graphs are the read-and-write events that are connected to each other via edges. These edges define the relationship and ordering among these events. For example, in place of saying that the value read by instruction c event in the Store litmus test comes from instruction b event, we can represent it as an edge from b to c [2, 3]. Our tests will focus on four kinds of edges:

- **Program-Order (po):** A program order edge specifies the order of events within a thread. If an event $E1$ is connected to another event $E2$ through po , then it implies that the $E1$ must appear before $E2$ in the execution of the program.
- **Write-Serialization (ws):** This edge is used to define the order of store events to the single global memory location.
- **Reads-From (rf):** A reads-from edge connects a write event $E1$ to a read event $E2$ if the value read by $E2$ is what was stored by $E1$. For example, events corresponding to operations b and c in the message passing test should be connected through the rf edge because the read event e_b generated by instruction b will read the value that comes from the write event, e_c , generated by instruction c .
- **From-Reads (fr):** A from-reads edge connects a read operation event $E1$ to a write operation event $E2$ such that the value read by $E1$ was written by a write operation event $E3$ that has a ws edge to $E2$. In other words, event $E3$ should have occurred before $E2$. This also implies that we can derive an fr edge from a ws edge and an rf edge [2]. The graph in Figure 2.8 depicts the relationship between an fr edge with an rf and ws edge. For example, in the Message Passing litmus test, d will have an fr edge with a for a weak behavior execution because it reads the value from the initialization write, which was made prior to a , and therefore will have a ws edge with a .

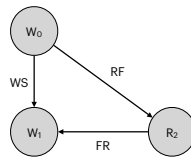


Figure 2.8: Deconstruction of an fr edge into a ws edge and rf edge.

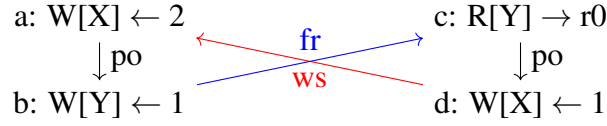


Figure 2.9: Graph representation of weak memory behavior execution for the Store Litmus test

All weak memory executions will be cyclic graphs. For example, Figure 2.9 shows the weak memory behavior execution for the Store litmus test using nodes and edges. Events *a* and *b*, and *c* and *d* have *po* edges representing that *a* should happen before *b* and *c* should happen before *d*. Event *b* has an *fr* edge to node *c* because event *c* reads the value stored by event *b*. For the weak memory outcome, the final value in *X* needs to be 2, which means the write event *a* must happen after the write event *d*, because of which *d* has a *ws* edge to *a*.

2.3 Empirical Testing

In practice, litmus tests are run for several iterations to observe weak behavior or reorderings. This is because every litmus test can have three outcomes: (1) sequential, where instructions/events from different threads are not interleaved (for example, executions 1 and 2 in Figure 2.1); (2) interleaved, where instructions/events from different threads are interleaved (for example, execution 3 in Figure 2.1), or (3) weak behavior. The weak behavior outcome happens as a result of instruction reordering, which is not guaranteed to happen in every test run/iteration. Hence, we need to run these tests numerous times to capture an execution that corresponds to a sequence reordering or a weak memory behavior [10]. We will focus on two testing methodologies in this paper:

2.3.1 Single Instance Testing

Single-instance testing has per-iteration synchronization. This synchronization is obtained by relaunching the test threads in every iteration. For example, in iteration i , we will launch the testing threads, perform the test, and then join the threads. After the threads are joined, we will check the values that were read during the test and the final values in the memory addresses getting read/written to check for a weak memory, sequential, or interleaved behavior. Then, for iteration $i + 1$, we will relaunch the threads again and repeat this process. The pseudo-code for single instance testing has been provided in Algorithm 1. This approach turns out to be costly, especially in heterogeneous testing, due to the overhead of relaunching and joining threads for every iteration. Tools like litmus7 implement a similar per-iteration synchronization approach for testing [8].

Algorithm 1 Single Instance Test Pseudocode

```
function TEST_THREAD_1()
    :
end function
function TEST_THREAD_2()
    :
end function
function MAIN
    for  $i = 0 \rightarrow N$  do
        INITIALIZEVALUES()
        LAUNCH_TEST_THREAD_1()
        LAUNCH_TEST_THREAD_2()
        JOIN_TEST_THREAD_1()
        JOIN_TEST_THREAD_2()
        OBSERVEVALUES()    ▷ Check conditions for weak mem.
        behavior
        :
    end for
end function
```

2.3.2 Perpetual Instance Testing

The perpetual instance testing approach [15] aims to reduce the overhead of re-launching kernels/threads by removing per iteration synchronization. In this approach, the threads are launched once, and we shift the for loop from the main function in Algorithm 1 to the testing threads. After all the iterations are completed, the threads are joined back. In our results (shown in Chapter 4), we show that this methodology is about 15,000 times faster than the single instance approach! This is because instead of relaunching the CPU/GPU threads 1 Million times (for every iteration), we just do it once.

As detailed in the PerpLE paper¹, to implement this approach, the actual test would have to be modified because we would not stop after every iteration to check for weak memory behavior. The PerpLE paper also discusses an example using the Store Buffer litmus test to illustrate the transition and differences between a single instance test and a perpetual instance test. Following this example, we describe the steps necessary for transitioning from a single instance test to a perpetual instance test for the Load Buffer litmus test (shown in Figure 2.7) below.

- In a single instance testing approach, $r1$ and $r0$ would be buffers of size 1 as they will store the value read in a particular iteration. We will reinitialize these buffers before every iteration. In perpetual instance testing, $r0$, and $r1$ would have to be buffers of size N (if there are N iterations), and index m in the buffer would correspond to the value read in iteration m of the test thread.
- For single instance testing, we can store constant values in every iteration (as

¹Throughout the thesis, we use the term ‘PerpLE’ to refer to the paper titled ‘PerpLE: Improving the Speed and Effectiveness of Memory Consistency Testing’ by Melissaris et al.

shown in Figure 2.7). However, we cannot do this for perpetual instance testing because we do not stop the threads after every iteration. Hence, if we keep storing constant values, then there would be no way to decipher the iteration in which a particular store was made [15]. For example, if we write constant values, then the load instructions would repeatedly read the value 1 and store them in the buffers, although every 1 might come from a different iteration. Unless we have a way to distinguish the iteration in which a store was made, we will not be able to infer whether we saw a sequential, interleaved or weak memory behavior. To resolve this, PerpLE suggests using increasing arithmetic sequences. Instead of storing 1 in instruction b , we can store $n_{T0} + 1$ where n_{T0} is the iteration number of thread 0, and instead of storing 2 in instruction d , we can store $n_{T1} + 1$ where n_{T1} is the iteration number of thread 1.

- The next step is to modify the conditions to check whether the resulting outcome corresponds to a weak, sequential, or interleaved behavior. For single instance tests, we would check the final values in the buffers $r0$ and $r1$ after an iteration is over. For perpetual instance tests, these checks would have to be transformed into linear equations using the information about the edges and the increasing values written in every iteration. For example, for the weak memory behavior condition, instead of checking $r0 = 1 \ \& \ r1 = 1$, we would have to iterate over the buffers and check if the following linear equation is true for any indices $n0$ and $n1$: $r0[n0] \geq n1 + 1 \ \& \ r1[n1] \geq n0 + 1$.

In Chapter 3, we explain how our testing framework handles these modifications. A big issue with the original perpetual instance approach is that it cannot be applied to tests that don't have 2 read instructions. This is because the heuristics or conditionals

for checking sequential/interleaved/weak memory behavior specifically rely on checking the values from the read buffers. Litmus tests, such as 2+2 Write, that do not have any read buffers, cannot utilize the proposed heuristics in the PerpLE paper. To extend the applicability of this approach, we propose a methodology called observer threads in Chapter 3 that allows us to use this approach for all kinds of litmus tests.

2.4 Heterogeneous Testing

In a heterogeneous architecture, different systems may allow for different memory reorderings that could even be disallowed on others, which is why testing for weak behaviors on shared memory of these heterogeneous systems becomes interesting! For example, the CPU may allow for a Load $\rightarrow$ Store reordering that could be disallowed on the integrated GPU. Our work focuses on exposing the behavior of shared memory in such cases.

We use the empirical testing methodologies defined in the prior section for testing heterogeneous systems. As we focus on CPU-GPU systems, we run one testing thread on the CPU and the other on the GPU. All the memory locations/addresses in the test (such as X and Y) are allocated on a shared memory array. Figure 2.10 visually shows the concept of heterogeneous testing.

2.5 Important Symbols and Terminologies

- **Nodes:** In the thesis, when we use the term ‘nodes’, it will refer to store/load operations/events. A store operation will be denoted by $W[X]$, where the term ‘W’ refers to ‘write’, and X is the memory location/address to which the

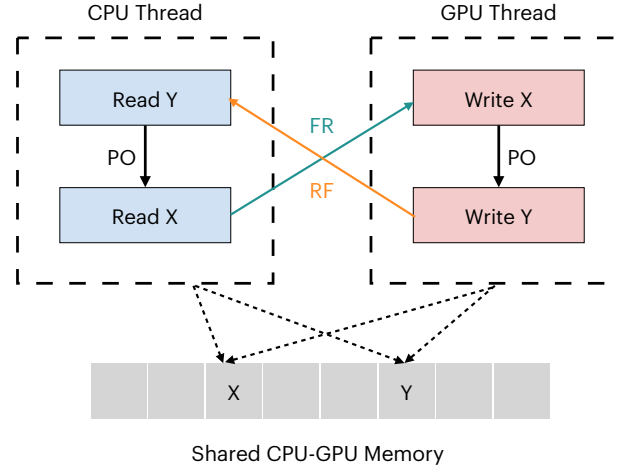


Figure 2.10: Litmus testing on heterogeneous CPU-GPU systems.

store is made. The value that is stored by the nodes is kept abstract and decided by the compiler. Load nodes are denoted by $R[X, r1]$ where the term ‘R’ refers to ‘read’, X is the memory location from where the value is read, and $r1$ is a temporary register/buffer where the value read is stored.

- **Edges:** Edges refer to one of the 4 edges (po, ws, rf, fr) discussed in Section 2.2.
- ρ_i : The symbol rho will be used to represent the value that is read by a read node. The subscript in rho is used to denote the order or the node-id of the read node. For example, ρ_1 means the value that was read by the read node with node ID = 1.
- ω_i : We will use this symbol to represent the value stored by a write node. The subscript denotes the order/ID of the write node. For example, ω_1 refers to the value stored by the write node with ID = 1.
- μ_i : Mu represents the ‘modulo-value’ that is assigned to every write node by our

compiler. The value stored by the write nodes in our tests is a function of the μ value and the iteration number of the store. Hence, the μ value acts like a unique identifier for the write node, which helps us in decoding the node ID that made a store. The process of assigning the μ values is discussed in detail in Chapter 3, which discusses the implementation of the compiler. We will often represent a write node like $W[X, \mu]$ to include the μ value associated to it.

Chapter 3

Compiler Design

This chapter discusses the compiler we built for automatically generating tests from the happens-before cyclic litmus test execution graphs. A compiler is needed because manually coding the litmus tests is both error-prone and time-consuming. Additionally, a compiler’s functionality can be easily extended to support tests for different systems or architectures, and it is useful in efficiently producing tests with varying parameters such as the number of threads, instructions, fences, etc.

Figure 3.1 shows the overall functionality of the compiler we built. Our compiler can generate three types of tests: (1) CPU-only litmus tests, (2) GPU-only litmus tests, and (3) Heterogeneous CPU-GPU litmus tests. In the CPU-only litmus tests, all the test threads run on the CPU, and in the GPU-only tests, all the test threads run on the GPU. In heterogeneous tests, half of the threads run on the CPU, and the other half run on the GPU, both utilizing shared memory. These tests can be generated in two configurations: single-instance and perpetual-instance. We have built a Vulkan and CUDA backend for tests that involve GPU threads. The CPU-only tests are produced in C++. These tests can have different fine-grained configurations that can be provided

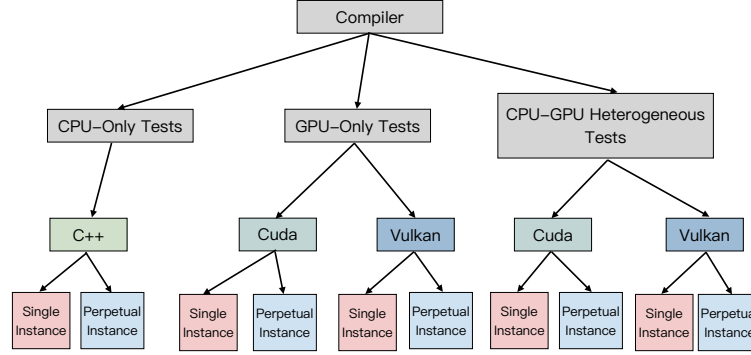


Figure 3.1: Flowchart representing different tests that can be produced by our compiler. At depth 2, the nodes show the type of tests we can generate. Depth 3 nodes show the backends we have for each of these test types, and depth 4 nodes show the different empirical testing methodologies we can use to generate these tests.

as input to the compiler and are covered in the section 3.2 of this chapter.

3.1 Algorithm at a Glance

Figure 3.2 shows a series of steps the compiler takes to generate the code for a litmus test. Below is a more fine-grained description of each step in the algorithm:

- a. In the first step, we parse the cyclic execution graph to the scanner and parser in the compiler. The scanner and parser have been implemented from scratch, only utilizing the Python regex library. Section 3.2 provides a comprehensive description of the compiler’s input, including the cyclic graph. The scanner generates three kinds of lexemes from the input graph: `NODE`, `EDGE`, and `COMPUTATION`. These lexemes are passed to the parser, which is responsible for creating instances of node and edge classes from the lexemes received. Then, the node and edge class instances are fed into our `TopLevelGraph` class instance. This `TopLevelGraph` class instance stores all the information about the test, such as

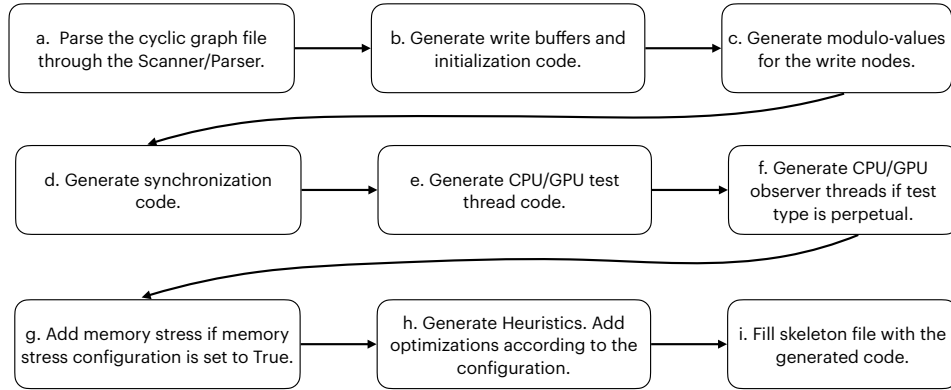


Figure 3.2: Coarse-grained view of the compiler algorithm.

the number of threads, list of nodes, list of edges, fences, number of observer threads, etc. This `TopLevelGraph` class instance will also store the code generated at each step in our algorithm.

- b.** After the first step, the compiler has all the information about the total number of threads, buffers, and variables that need to be defined. In the second step, the compiler creates the initialization code for the temporary variables and buffers. For example, if we are producing a heterogeneous test in CUDA, then all the calls to `cudaAlloc` and `cudaFree` for the shared variables will be generated in this step.
- c.** In the third step, we iterate over all the write-nodes and generate modulo-values (μ) for them. This step has been comprehensively explained in section 3.4, which also explains how this modulo-value is utilized during the test. In actuality, μ values are only required for perpetual tests, but for symmetry, we generate and use them for single instance tests as well.

- d. After assigning the μ values to the write nodes, we generate the synchronization code in step four. The testing threads should ideally start at the same time for there to be interesting system interleavings in memory operations. To achieve this, we add a light-weight barrier in the testing threads before an iteration begins. For single-instance tests, this synchronization/barrier is added for every test iteration, which also adds to the overhead. For perpetual instance tests, we add this synchronization before the for loop in the test threads - our aim is to make the for loops start at the same time. The iterations can become desynchronized later on in the test, and this issue has been addressed in the sections later on. For this barrier synchronization, we use an atomic integer that is incremented by every thread. After incrementing this integer, the threads spin till the value of the atomic integer becomes equal to the number of threads (indicating all threads are ready to begin). Figure 3.3 shows how this synchronization works in a system where we have 3 testing threads.
- e. The next step is to generate the main CPU/GPU thread code, which consists of generating code for the nodes that were specified in the execution graph. In this step, the compiler generates the CPU/GPU function headers, function body and calls to them from the main function. Every CPU test thread function name is of the format `test_function_i` where i is the thread ID specified in the graph input file. On the GPU, the function names are of the format `test_function_gi` where i is the GPU thread ID specified in the graph input file. For every thread, there are 2 kinds of code generated: pre-sync and post-sync. The pre-sync code consists of stores/load optimization discussed in section 3.6. In the pre-sync code, we extract the address of the memory locations the test stores/loads from,

so we don't have to do it in every iteration. The post-sync code is where the node operations happen. The final body of the function is: pre-sync + sync code generated in step d + post-sync code.

- f.** After the CPU/GPU threads are generated, the compiler generates the code for observer threads. Observer threads are only required for perpetual instance tests, and we added them to extend the applicability of the original perpetual testing approach. Section 3.3 defines the purpose and logic behind them and how we use them to generate heuristics/conditions for checking weak memory behaviors. The number of observer threads is specified as an input to the compiler through a configuration file (see Sec. 3.2).
- g.** In the next step, the compiler generates the memory stress code if its flag has been set to true in the configuration file. In heterogeneous tests, we add memory stress on both CPU and GPU; in CPU-only and GPU-only tests, all stress threads are on the CPU and the GPU, respectively. The purpose of adding this stress is because weak memory behavior is often prone to memory stress. Section 3.6 talks about the need for memory stress, Chapter 4 presents results that highlight the behavior of memory stress in single instance tests and perpetual instance tests.
- h.** In the second last step, the compiler adds the trace-analysis heuristics. Our compiler generates heuristics (conditions) for detecting weak memory behavior using the node-edge relations from the input graph. The methodology for generating heuristics for single instance tests is different from the methodology we use for generating perpetual instance tests, and section 3.5 discusses them, including the optimizations we added for reducing their time complexity.

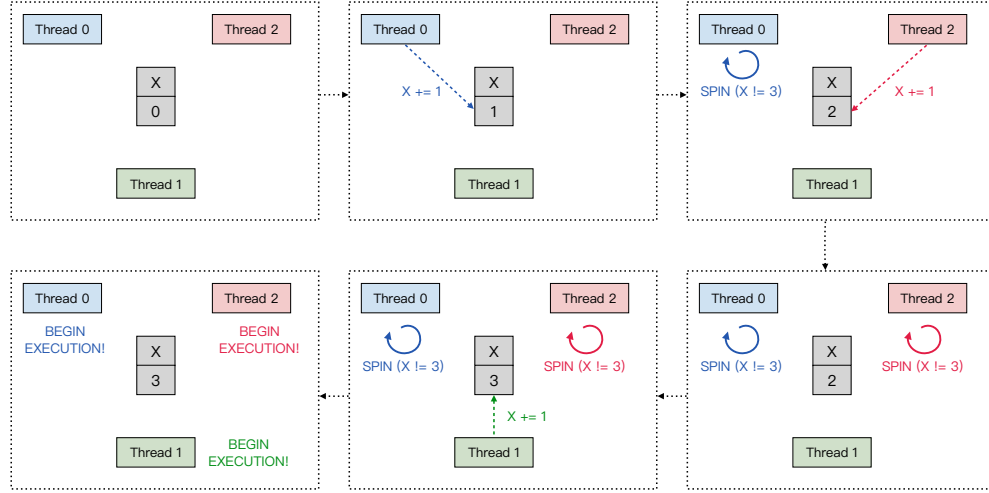


Figure 3.3: Barrier synchronization example using three test threads.

- i. In the final step, we stitch the code we generated in the previous steps into the template files. The templates/headers consist of common code, such as library imports, `cudaAllocMapped` functions, etc, that will be the same for every test regardless of the configuration we choose. Hence, instead of letting the compiler generate this common code for every test, we put it in a template file and only generate components that will be unique for every test using our compiler.

3.2 Compiler Input

The input to the compiler is provided through a .JSON file that defines the fine-grained configurations and the paths to the cyclic execution graph and other template files. Listing 3.2 provides a sample .JSON configuration file. The subsections below describe all the components in the configuration file and the cyclic execution graph.

3.2.1 Cyclic Execution Graph

The cyclic graph discussed in Section 2.2 is provided to the compiler as a text file. The text file has three components to it:

- **Nodes:** The nodes are specified with a unique node ID and can either be write operations or read operations. If a node is a read operation, it also specifies a temporary buffer/register in which the value read should be stored. For example, $N1 = W[X]$ means a write node that operates on the memory location X and has the Node ID $N1$. Similarly, a read node can be defined as $N2 = R[X, r1]$ where X is the memory location the node reads from, $r1$ is the temporary buffer in which the value read is stored, and $N2$ is the unique node ID.
- **Edges:** The edges can be of types: po , ws , rf , or fr . The definitions of the edges have been provided in Section 2.2. In the text file, the edges are specified after the nodes and have the format $e : N_x \rightarrow N_y$ where $e \in \{po, ws, rf, fr\}$ and N_x and N_y are unique node IDs.
- **Computation:** This part of the input specifies which nodes are supposed to be executed on the CPU and/or the GPU. Nodes computed in the same thread are separated by a colon. For example, $CPU_Tn = N_x; N_y$ specifies that nodes with IDs N_x and N_y should be computed on the CPU thread with ID n in the specified order. Similarly, we can create GPU computation threads. We can also specify fences in the CPU and GPU nodes. For example, specifying: $CPU_T1: N1; fence; N2$ would add an atomic fence between instructions $N1$ and $N2$ on the CPU thread.

Listing 3.1 shows the graph text file for the Store Buffer test that can be provided as

input to the compiler. All the nodes are defined first, then the edges, and finally, the computation. In this example, nodes $N1$ and $N2$ are executed on a CPU thread, and nodes $N3$ and $N4$ are executed on a GPU thread. If we had more nodes in our test, then we could have computed them in different CPU and GPU threads as well by specifying GPU_T2, CPU_T2, and so on.

```

1  N1 = W[X]
2  N2 = R[Y, r0]
3  N3 = W[Y]
4  N4 = R[X, r1]
5
6  po: N1 -> N2
7  po: N3 -> N4
8  fr: N2 -> N3
9  fr: N3 -> N4
10
11 CPU_T1: N1; N2
12 CPU_T2: N3; N4

```

Listing 3.1: Execution graph file the Store Buffer litmus test.

3.2.2 Fine-Grained Configurations

The configuration file has 12 parameters that specify the fine-grained configuration of the test produced. Below is a description of the parameters:

- **type:** This parameter can be either set to ‘single’ for generating a single instance test or to ‘perp’ for generating a perpetual instance test.

- **lang**: This parameter is used for specifying the backend that should be used for the test. For CPU only tests, this parameter carries lesser meaning because we generate them in C++ only. However, for GPU-only and heterogeneous tests, this parameter can be either set to ‘Vulkan’ or ‘Cuda’.
- **cpu_obs**: This parameter is only relevant for perpetual instance tests because there is no concept of observer threads in single instance tests, and is used for specifying the number of observer threads on the CPU side. If the test type is GPU-only, then this parameter should be set to 0.
- **gpu_obs**: As in the case of `cpu_obs`, this is only relevant for perpetual instance tests and is used for specifying the number of observer threads on the GPU. If CPU-only tests are being produced, then this parameter should be set to 0.
- **mem_stress**: Setting this parameter/flag to true would allow the compiler to add memory stress functionality in the test files. We will expand on the benefits of adding memory stress and its impact on the overall weak memory throughput in later sections.
- **window**: This parameter is used to specify the window size that is needed for producing the heuristics with optimizations. The compiler produces a set of heuristics for checking whether a weak memory behavior was observed at any point during the test. This size specifies the spatial closeness of the indices getting compared from buffers on different threads. We discuss this in more depth in section 3.5.
- **ws_pruned**: Setting this to true would reduce the heuristic time complexity for checking a *ws* edge and also remove any redundant conditionals from the heuris-

tics that are associated with the *ws* edge. The functionality/optimization added by this flag is discussed in Section 3.5.

- **po_pruned**: Setting this to true would reduce the heuristic time complexity for checking a *po* edge if it connects 2 read nodes and would also remove any redundant conditionals from the heuristic that are associated with checking for this edge. The functionality/optimization added by this flag is discussed in Section 3.5.
- **prune_buffers**: Turning this parameter to true would produce pruned buffers from the observer thread buffers. When we record values from the observer threads, then we can end up reading the same value in multiple iterations. Because of this, there might be duplicate values in the adjacent indices. Setting this parameter to true would add functionality in the generated files to remove all the duplicate values from the observer buffers.
- **optimized_heuristics**: We produce two kinds of heuristics for the perpetual tests. Setting this parameter to true would generate the optimized version of them that have time complexity $O(N)$.
- **mode**: This parameter is used for specifying whether the test produced should be CPU-only, GPU-only, or heterogeneous.
- **mem_order**: This parameter is for specifying whether the atomic read/write operations on the CPU should be relaxed or not. For example, if the `mem_order` is set to 'relaxed'. then all the atomic loads/stores will have `memory_order_relaxed`.

3.2.3 Template Files and Output Files

The .JSON configuration file also specifies the path of the template files and the output files. While building these tests, we realized that some parts of these files would always be the same, no matter the configuration. For example, every CUDA test would import the same libraries, would have some common variables, and would have the same print statements at the end, etc. Therefore, instead of regenerating the common/overlapping code, we include it in a template file and only generate the parts that would be unique for the tests.

However, the structure of a single instance test file would be different from a perpetual instance test file, and the overlapping code in the CUDA test files would be different from the Vulkan test files. Because of this, we do not have a global skeleton/template file. Instead, we have 6 template files:

- CUDA template file for single instance tests
- Vulkan template file for single instance tests
- CUDA template file for perpetual instance tests
- Vulkan template file for perpetual instance tests
- C++ template file for CPU-only single instance tests
- C++ template file for CPU-only perpetual instance tests

The first four files can be used to generate both heterogeneous tests and GPU-only tests. The only difference is that in the GPU-only tests, there would not be any CPU threads.

```
1      {
2      "configuration": {
3          "type": "perp",
4          "lang": "cuda",
5          "cpu_obs": 1,
6          "gpu_obs": 1,
7          "mem_stress": false,
8          "window": 5,
9          "ws_pruned": true,
10         "po_pruned": true,
11         "prune_buffers": false,
12         "optimized_heuristics": true,
13         "mode": "HETEROGENEOUS",
14         "mem_order": "relaxed"
15     },
16
17     "files": {
18         "template": "skeleton_perp_hetero.cu",
19         "output_file": "sample.cu",
20         "graph": "graphs/graph.txt",
21         "cl_filename": "sample_cl.cl"
22     }
23 }
```

Listing 3.2: Sample .json configurations file for the compiler

3.3 Observer Threads

The original perpetual instance approach is applicable to tests that have at least 2 read instructions, such as Store Buffer, because they rely on checking the values from the read buffers for inferring a weak memory behavior. The read buffers, as explained in the previous sections, store the values read by the nodes in every iteration. However, we do not store the final values of the memory addresses (such as X and Y) after every iteration. During our experiments, we realized that adding read operations at the end of each iteration to record the final values of the memory addresses can end up interfering with the litmus test itself. Hence, there is no way to infer ws and po edges that don't have any read nodes with the original approach.

To solve this issue and extend the perpetual testing approach to handle different litmus tests, we introduce the concept of 'observer threads'. In a nutshell, the idea is to continuously snoop on the values stored at the memory locations/addresses during the test execution and create a global timeline that can help us see how the values changed over time during the iterations. Seeing these changes can help us unmask the order in which the store operations were processed, allowing us to infer the ws and po edges.

An observer thread has a simple structure. It reads from a memory location repeatedly and stores the values in a buffer called the 'observer buffer'. Every variable or memory location has its separate observer buffer. In our tests, we have multiple observer threads to record as many events/unique values as possible, and the number of observer threads on the CPU and the GPU can be specified in the config file for the compiler. Based on these values, the compiler creates observer buffers for each variable and for each observer thread.

$$\forall m \in M, Total(Obs_m) = OBS_{CPU} + OBS_{GPU} \quad (3.1)$$

$$\sum_{m=0}^n Total(Obs_m) = (OBS_{CPU} + OBS_{GPU}) * (|M|) \quad (3.2)$$

In Equations 3.1 and 3.2, M is the set of memory locations in a test, OBS_{CPU} is the number of CPU observer threads, and OBS_{GPU} is the number of GPU observer threads. $Total(Obs_m)$ represents the total number of observer buffers for a memory location m .

Algorithm 2 shows the pseudo-code for an observer thread. In this code, we repeatedly load the values from all the memory locations involved in the litmus test, from X_1 to X_m in our example, and then store them in the respective buffers. The ‘SYNCHRONIZE’ before the for-loop is added because we want to start the observer threads at the same time as the main test threads so we can record maximum unique values that span throughout the test iterations. This synchronization will follow the same methodology as described in section 3.1.

Algorithm 2 Observer thread pseudo code

```

SYNCHRONIZE
for  $n = 1$  to total_iterations do
   $obs_{X_1}[n] \xleftarrow{Load} X_1$ 
   $obs_{X_2}[n] \xleftarrow{Load} X_2$ 
   $\vdots$ 
   $obs_{X_m}[n] \xleftarrow{Load} X_m$ 
end for

```

In our heterogeneous tests, we have observer threads on both the CPU and the GPU. Figure 3.4 shows the functioning of these observer threads logically in a small test where we have two memory locations/variables that get read/written to by the main test

threads. The thin yellow bar over each of the threads represents synchronization, so all the threads begin at the same time.

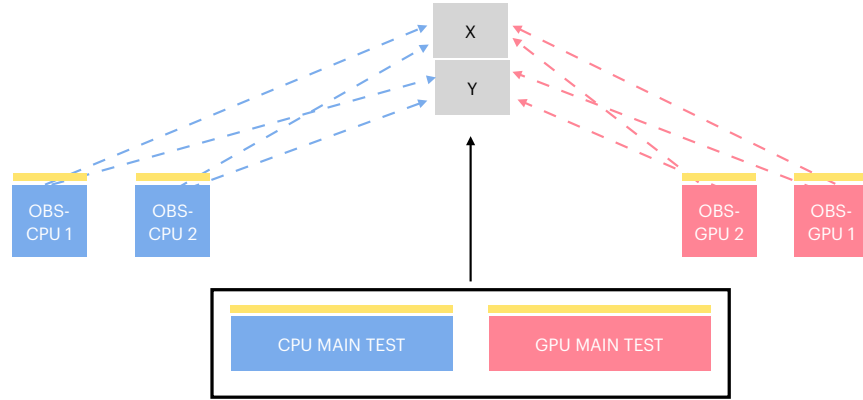


Figure 3.4: Example demonstrating observer threads. The yellow bar on top of each thread represents the barrier synchronization. X and Y are locations/addresses on shared memory.

Once we have the timelines/observer buffers at the end of the tests, we use them to check whether we saw any weak behaviors using the heuristics described in Section 3.5 of this chapter.

3.4 Choosing Values for the Write Nodes

As could be observed from Section 3.2.1, we do not specify the values that should be written by the write nodes in our cyclic graph file. This is to avoid specifying a constant write value. While for single-instance tests, writing constant values is fine because we reinitialize the variables before every iteration, constant values become a problem with perpetual instance tests. This issue has also been highlighted in the PerpLE paper with the solution of writing arithmetic sequences [15].

When designing our arithmetic sequences for perpetual tests, we had to address two complexities that arise because of observer threads:

- If multiple nodes are writing the same values in the memory locations, then we would not be able to decipher which node a value comes from. The solution for this issue is to assign unique values to every write-node, so when we look at the timeline/buffers generated by the observer threads, we can understand what node the value came from. However, this solution leads us to our second problem.
- If we write constant unique values in every iteration of perpetual tests, then looking at the timeline would give us no indication about when in time (iteration) a value was written by the node. The solution for this is to write increasing values in every iteration. This way, we can decipher the iteration in which a value was written by seeing how large it is.

Therefore, to address both issues, we write increasing unique values in the write-nodes. To do this, the compiler first chooses a constant K such that $K = 1 + \text{Number of Nodes}$. Every write node gets assigned a μ called the modulo-value such that $0 < \mu < k$. The value written by the n th write-node will be $(\text{iteration_count} * K) + \mu_n$ where μ_n is its modulo-value. We use this equation because doing a mod operation with the constant K can give us information about the node that wrote a value. Similarly, we can also easily decipher the iteration in which a value was written by subtracting μ_n and dividing by K . Our arithmetic sequence is quite similar to the one used in PerpLE [15] with some nuanced logical differences. For example, consider the Store Buffer test, which has also been discussed in the PerpLE paper. The arithmetic sequences written by PerpLE are shown in Equation 3.3, and the arithmetic sequences that our compiler generates for

the store nodes are shown in Equation 3.4¹.

$$\begin{aligned} W[X] &\rightarrow T0_n + 1 \\ W[Y] &\rightarrow T1_n + 1 \end{aligned} \tag{3.3}$$

$$\begin{aligned} W[X] &\rightarrow (5 * T0_n) + 1 \\ W[Y] &\rightarrow (5 * T1_n) + 2 \end{aligned} \tag{3.4}$$

The problems we discussed above do not affect single-instance tests. This is because we stop at every iteration to check for weak memory behavior. However, we still write increasing unique values in every iteration of the single-instance tests to ensure symmetry in the approach with perpetual-instance tests.

Algorithm 3 Unchanged test: writing non-unique and non-increasing values

CPU Thread:

```

for  $i = 1$  to total_iterations do
   $x \leftarrow 1$ 
   $y \leftarrow 2$ 
end for

```

GPU Thread:

```

for  $i = 1$  to total_iterations do
   $y \leftarrow 1$ 
   $x \leftarrow 2$ 
end for

```

Algorithm 4 Our Approach: writing unique and increasing sequences

CPU Thread:

```

for  $i = 1$  to total_iterations do
   $x \leftarrow (K * i) + 1$ 
   $y \leftarrow (K * i) + 2$ 
end for

```

GPU Thread:

```

for  $i = 1$  to total_iterations do
   $y \leftarrow (K * i) + 3$ 
   $x \leftarrow (K * i) + 4$ 
end for

```

¹In the example shown, the μ value for the first write node is 1, and the μ value for the second write node is 2. However, this μ depends upon the order in which nodes are specified in the graph input file. For example, if the node $W[Y]$ was the fourth node in the list of nodes specified in the input file, then its μ value would have been 4.

3.5 Generating Trace Analysis Heuristics

In the single instance tests, after each iteration, the final values in the memory addresses and in the read buffers act as ‘traces’ that can help us infer whether we had a sequential, interleaved, or weak memory behavior execution. For example, if we have the Store litmus test (see Sec. 2.1.2), and the value at memory address X is 2 and the value in the buffer $r0$ is 1 after an iteration, then it means we had a weak memory behavior execution in that iteration. Similarly, for the perpetual instance tests, the observer buffers and the read buffers act as ‘traces’, and analyzing them can help us infer what kind of execution happened at an iteration during the test and whether we saw any weak memory behavior. In the sections below, we describe the heuristics and techniques we use to analyze these traces for single instance tests and perpetual instance tests.

3.5.1 Single Instance Tests

For generating the heuristics for single instance tests, we do a graph traversal on the execution graph. During the graph traversal, we keep track of how the values of memory locations change based on the read and write nodes that operate on them. Once we complete the traversal, the final values inferred for the memory locations would be the ones that correspond to a weak memory behavior. Hence, after an iteration of the single instance test is done, and the values of the memory locations are what we inferred from doing the graph traversal, then that iteration corresponds to a weak memory behavior. Another important aspect that we modify as we do the graph traversal is whether we need to check the memory location or one of the buffers that contain the value read from that location during the test. For example, we can have a variable/memory location in a litmus test, Y , that corresponds to the index i_m in an array, ‘com’, and one of the nodes

can be $R[Y, r1]$ in the test. Now, for inferring a weak memory behavior, we can either check $\text{com}[i_m]$ or we can check the value in the buffer $r1$ after the test is over. Hence, we will keep track of two parameters for every memory location/variable during the traversal:

- Final value after a node is finished.
- Whether the value stored in the read-buffer or the memory location would be evidence for a weak memory behavior ².

We have different rules for different cases (combinations of types of nodes and edges) for changing the variable values and evidences:

- **Cases for read-node:**
 - $R[X] \rightarrow r1$ is the child_node of an rf edge: We will make 2 changes in this case. First, we will update the evidence for X to the buffer $r1$. Second, we will change the value for X to ρ_0 such that $\rho_0 = \omega$ where ω is the value written by the parent write node of the rf edge.
 - $R[X] \rightarrow r1$ is the parent_node of an fr edge: We will make 2 changes in this case as well. First, we will update the evidence for X to the buffer $r1$, as in the previous case. Then, because this is the parent node for an fr edge, the value read by this node should come from the writes that happened prior to its child write node. Hence, we will search for nodes that

²We specifically mention that we need to check the evidence (memory address or read buffer) to infer weak memory behavior because our primary goal through these tests is to observe weak behaviors. However, if the execution graph provided to the compiler corresponds to a sequential execution, then checking the evidence for the final value would help us infer a sequential behavior instead of a weak memory behavior.

are connected to the child write node through ws edges. There can be two cases:

- * The child write node does not have any incoming ws edges: In this case, the value that was written prior to this write must be the initialization value. Hence, we will set the value of the variable to 0.
- * The child write node has incoming ws edges: In this case, the value in $r1, \rho$, could either be the initialization value or the value written by any of the parent nodes of the incoming ws edges:

$$\begin{aligned} (R[X, r1] \xrightarrow{fr} W_\alpha[X]), ((W_0[X] \leftarrow \omega_0) \xrightarrow{ws} W_\alpha[X]), \dots, \\ ((W_n[X] \leftarrow \omega_n) \xrightarrow{ws} W_\alpha[X]) \implies \rho \in \{init, \omega_0, \dots, \omega_n\} \quad (3.5) \end{aligned}$$

Hence, in this case, we will provide a set of values indicating that the final value in $r1$ could belong to any of the values in the set.

- **Case for write-node:**

- $W[X] \leftarrow \omega_0$: In this case, we will update the value for X to ω_0 indicating that during the test, after this node has been executed, the value in memory location X would be ω_0 . However, if the evidence for X is a read register, then this update will not happen. We will only make this update if the evidence for the weak memory behavior is the memory location. We have not split this into different cases, because the changes we will make on seeing a write node would be the same regardless of the incoming/outgoing edges.

We show 2 examples of this graph traversal process. Figure 3.5 shows this graph

traversal on the 2 + 2 Write litmus test. In step 0, we have the value attributes for X and Y initialized to 0 and the evidence attributes set to X and Y , respectively. We start the graph traversal by the write node storing 1 to X . Because this is a write node, we do not change the evidence for X but update the value attribute to the store made by the write node (1 in this case). Then, in the second step, we move to the next node, which does a store on Y . As in the previous step, we update the value attribute of Y to 2 (value stored by the write node). The evidence for Y remains unchanged. Similarly, we update the values for X and Y in steps 3 and 4. In step 5, we traverse the first node again to process the updates made by the last ws edge and update the value attribute of X to 1. The traversal is completed after step 5, and we get the conditions for weak memory behavior. Therefore, after a test iteration is completed, and the values in memory locations X and Y are 1 and 3, it implies there was a weak memory behavior.

Figure 3.6 shows the graph traversal on the Store litmus test. As in the 2 + 2 write test example, we initialize the value attributes for all the variables to 0 and the evidence attributes to the memory locations (X and Y), respectively. We start the graph traversal by iterating over the first write node that stores 1 to X . The updates in steps 1 and 2 are made similar to as in the previous example. In step 3, we process the read node. Hence, we update both the evidence and the value attribute of Y . The evidence is updated to the read buffer $r0$, and because we have an rf edge, we update the value based upon the parent node of the rf edge, which is the write node that stores the value 2. The dotted arrows in step 3 show what nodes are responsible for updating the value and evidence attributes of Y . In step 4, we process the write node and update the value attribute of X to 4. As in the previous example, in step 5, we traverse the first node again to record the changes made by the last edge and update the value attribute of X back to 1. After step

5, our graph traversal is completed, and if after a test iteration is completed, the value in buffer $r0$ is 2 and the value in the memory location X is 1, then it implies we saw a weak memory behavior.

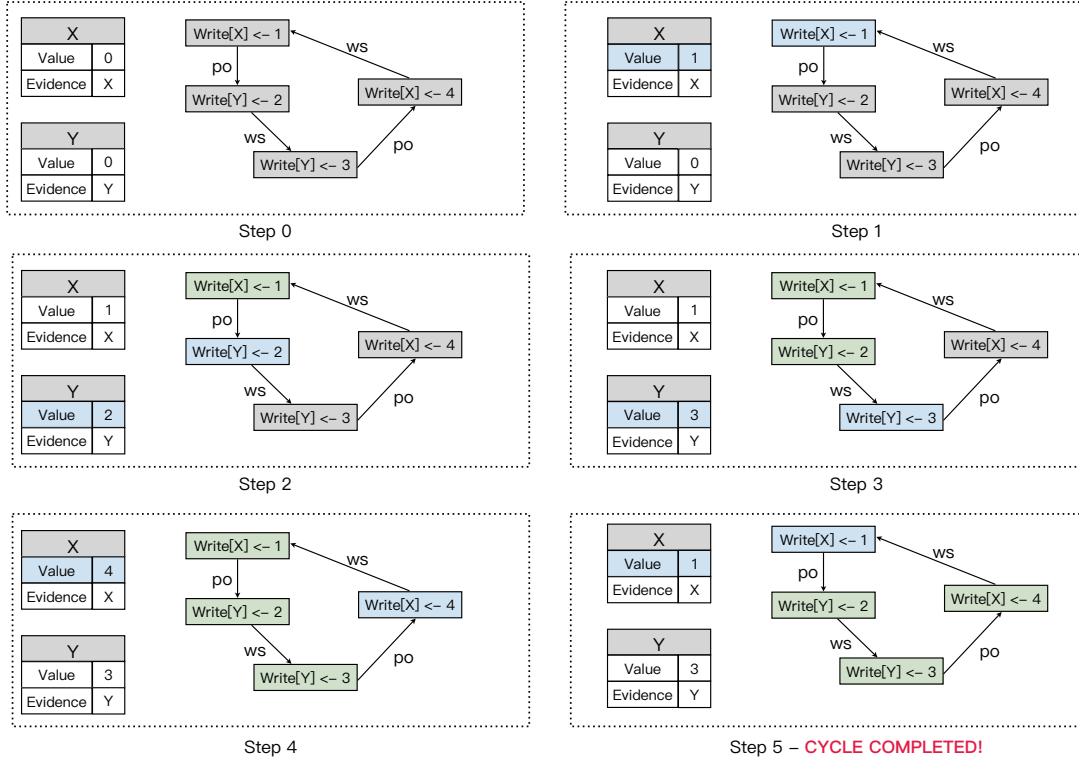


Figure 3.5: Graph traversal on the 2 + 2 Write litmus test execution graph to determine conditions for weak memory behavior.

3.5.2 Perpetual Instance Tests

Generating heuristics for perpetual instance tests happens to be much more complex than a simple graph traversal because we need to analyze weak memory behaviors using the long traces we generated from the observer threads. Our heuristics are different

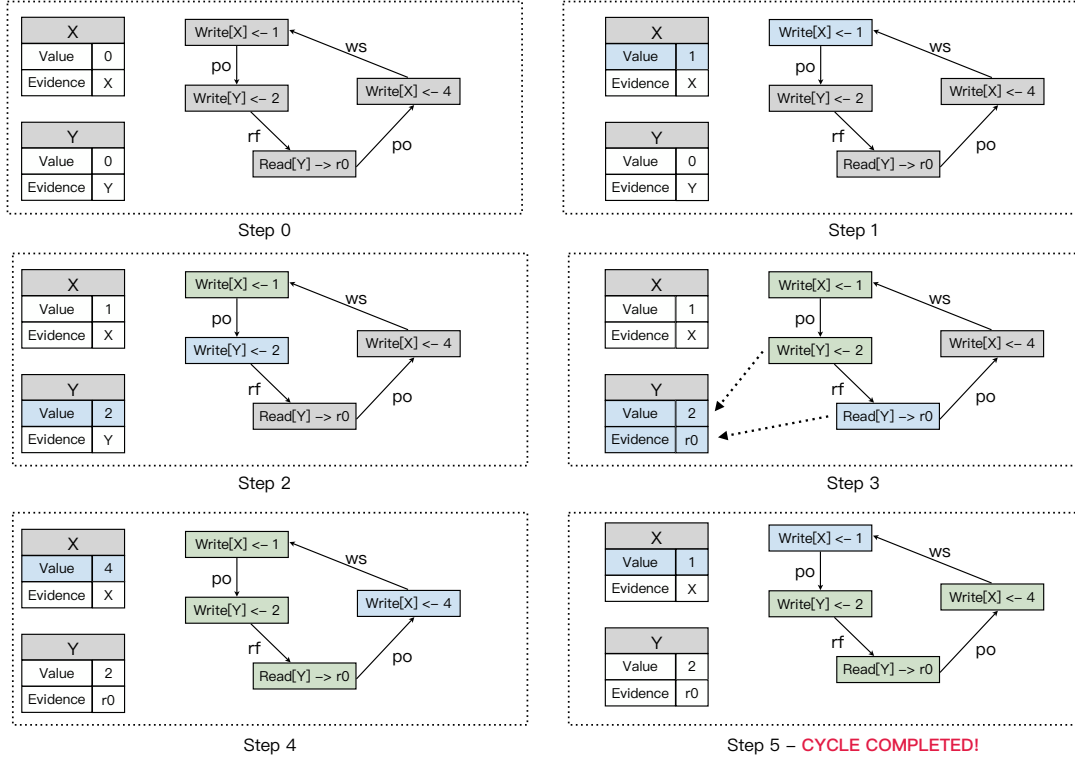


Figure 3.6: Graph traversal on the Store litmus test execution graph to determine conditions for weak memory behavior.

from the perpetual instance test heuristics proposed in PerpLE [15] because of observer threads and because we want to generalize these to be applicable to any litmus test and not just the ones that only have read buffers.

First, we iterate over the read buffers and the observer buffers to get the values that correspond to all the nodes. For getting values of write nodes, we need to iterate over the observer buffers and do modulo checks to confirm the source node. For example, if there are N nodes that are writing to the same memory location X , then by our methodology described in section 3.4, every node will be assigned a modulo-value that will function as a unique identifier for the values written by that node. Hence, if we want

to check whether a value read by an observer thread from the memory location X was written to by Node N_1 , then we need to mod the value by the constant selected earlier and check whether we get node N_1 's assigned modulo-value. For getting a value associated with a read node, we have to iterate over the node's read buffer, and no conditional check is required as in the case of write nodes. A read node operating/reading from a memory location X is allowed to read any value written to by write-nodes that operate on X .

Then, we analyze the relation between the values to check whether we can reconstruct the critical cycle in the litmus test graph. If there is a cyclic relationship between the values then it means we saw a weak memory behavior. Hence, our heuristics will have a conditional check for each edge in the test.

For example, assume we have the following nodes that form a cycle: $N_1, N_2, \dots, N_m$ such that the last node N_m connects back to the first node N_1 through some edge $e \in \{po, rf, fr, ws\}$. Now, to check for a weak memory behavior, we will iterate over the observer buffers (for the write nodes) and the read buffers (for the read nodes) to get a value corresponding to every node. For write nodes, this value would be the value written to a memory location by them that was read by the snooping observer threads, and for the read nodes, this value would correspond to the value read by the node during the test. Hence, we will have values: $\{v_1, v_2, v_3, \dots, v_m\}$ such that the value v_k corresponds to N_k . The next step would be to check whether we have the edges in between these values that were specified in the test to form a cycle.

$$V = \{(v_1 \in N_1), \dots, (v_k \in N_k)\} : \forall (v_x, v_y \in V), N_x \xrightarrow{e} N_y \implies v_x \xrightarrow{e} v_y \quad (3.6)$$

The equation above defines this logic. If all the values have the same relationships

amongst themselves as the nodes they are associated with, then it implies we have a weak memory behavior because we have the cyclic relationship! For example, if values v_x and v_y come from nodes N_x and N_y , respectively, such that N_x and N_y have a program-order edge, then we need to check for a program-order relation between the values v_x and v_y . If there is a valid program-order relation, then it means we have validated this edge in the graph. If we can validate all the edges in the cycle, then it means we saw a weak behavior.

Below, we describe the heuristics/methodology we use to validate all the four edges:

Checking for the Program-Order Edge: We can have 4 cases when checking for a program-order edge between 2 values. This is because both the values can either come from a write node or from a read node, giving us 4 different configurations:

- $W_0[X_i, \mu_0] \xrightarrow{po} W_1[X_j, \mu_1]$: Both the nodes are write nodes. The first node has the modulo-value μ_0 , and the second node has the modulo value μ_1 . The nodes operate on memory locations X_i and X_j , respectively, which may or may not be the same. In this case, both the values will be extracted from the observer buffers. As we already write increasing values for the nodes, if W_1 comes after W_0 , then it means the compiler would have assigned W_1 a higher modulo value than W_0 , which implies that the values written by node W_0 would be $<$ values written by node W_1 for a particular iteration.

$$\forall i \in [0, \text{iterations}], W_0 \xrightarrow{po} W_1 \implies \mu_0 < \mu_1 \implies \omega_0 = (i * K) + \mu_0 < \omega_1 = (i * K) + \mu_1 \quad (3.7)$$

Hence, to check for a *po* edge between ω_0 and ω_1 , we need to check the following

condition:

$$\begin{aligned}
& \forall n \in obs_{X_i}, \forall m \in obs_{X_j}, (obs_{X_i}[n] \bmod K = \mu_0) \\
& \wedge (obs_{X_j}[m] \bmod K = \mu_1) \wedge (obs_{X_i}[n] < obs_{X_j}[m]) \implies obs_{X_i}[n] \xrightarrow{po} obs_{X_j}[m]
\end{aligned} \tag{3.8}$$

Breaking down the above equation:

- $(obs_{X_i}[n] \bmod K = \mu_0)$: Confirms whether the value in $obs_{X_i}[n]$ comes from node W_0 .
- $(obs_{X_j}[m] \bmod K = \mu_1)$: Confirms whether the value in $obs_{X_j}[m]$ comes from node W_1 .
- $(obs_{X_i}[n] < obs_{X_j}[m])$: Checks whether there is a po edge between both the values.
- $R_0[X_i, r0] \xrightarrow{po} R_1[X_j, r1]$: Both the nodes are read nodes. The first node reads the values from the memory location X_i and stores them in buffer $r0$, and the second node reads the values from the memory location X_j and stores the values in buffer $r1$. In this case, both the values will be extracted from the read buffers. With read nodes, we would not have modulo checks because the read nodes are not assigned any unique modulo-values.

Consider that the value read by the first read node is ρ_0 , and the value read by the second read node is ρ_1 . Comparing which value is bigger would not give us any information about whether there is a program order edge between the read nodes. However, the indices at which these values were read can give us crucial information. A program order edge between R_0 and R_1 means that R_0 should

have happened before R_1 . This gives us the following equation:

$$\forall n \in r0, \forall m \in r1, n \leq m \implies r0[n] \xrightarrow{po} r1[m] \quad (3.9)$$

Hence, any read made at an index n will have a po edge with all the reads made at index $\geq n$.

- $W_0[X_i, \mu_0] \xrightarrow{po} R_1[X_j, r0]$: The first node is a write node operating on the memory location X_i , and the second node is a read node operating on the memory location X_j . X_i may or may not be the same as X_j . μ_0 is the unique modulo-value that has been assigned to the write node. In this case, the first value will be extracted from the observer buffers, and the second value will be extracted from the read buffer $r0$. Now, because the read node is allowed to read any valid value stored in X_j , making a value comparison would be futile. Instead, the indices in which the read and write happened can give us important information about when in time the two instructions took place, helping us to infer the po edge. If the index at which the write happened is $\leq$ index at which the read happened, then there is a valid po edge. We can translate this reasoning into the following equation:

$$\forall n \in r0, \forall m \in obs_{X_i}, (obs_{X_i}[m] \bmod K = \mu_0) \wedge \left(\frac{(obs_{X_i}[m] - \mu_0)}{K} \leq n \right) \implies obs_{X_i}[m] \xrightarrow{po} r0[n] \quad (3.10)$$

Breaking down the above equation:

- $(obs_{X_i}[m] \bmod K = \mu_0)$: Confirms whether the value in $obs_{X_i}[m]$ comes from node W_0 .

– $(\frac{(obs_{X_i}[m] - \mu_0)}{K} \leq n)$: The value that comes from the write node is $obs_{X_i}[m]$.

This write node has the modulo-value μ_0 , which means the first term in this expression will give us the index in which the write happened. Then we check whether this index is less than or equal to the index in which the read happened, which is simply n .

- $R_0[X_j, r0] \xrightarrow{po} W_1[X_i, \mu_0]$: This case is symmetrical to the above case. The only difference in the equation for this case will be in the right conjunct, where we will check that the index of the read is less than or equal to the index of the write. The equation for this case will be:

$$\forall n \in r0, \forall m \in obs_{X_i}, (obs_{X_i}[m] \bmod K = \mu_1) \wedge (n \leq \frac{(obs_{X_i}[m] - \mu_1)}{C}) \implies r0[n] \xrightarrow{po} obs_{X_i}[m] \quad (3.11)$$

Checking for the Write-Serialization Edge: Unlike for the *po* edge, we cannot have different cases for the *ws* edge. If a node $N1$ is connected to another node $N2$ through the *ws* edge, then it implies that both nodes $N1$ and $N2$ are write-nodes and write to the same memory location. Their modulo-values will, however, be different. Hence, the only case we will have for this edge will be: $W_0[X_i, \mu_0] \xrightarrow{ws} W_1[X_i, \mu_1]$. To check for this *ws* edge, both the values for the nodes will have to be extracted from the observer buffers. We need to check that the write W_0 happened before the write W_1 , which is similar to checking that we observed that value written by node W_0 (ω_0) before we observed the value written by node W_1 (ω_1).

Usually, we will have multiple observer threads that read from the same memory location to record as much data as possible. These different threads might not run in

total synchrony, which is why we cannot compare the indices of observer buffers from two different observer threads to infer which observation happened first in time. Hence, we will have to use the observer buffers from the same observer thread to check for the *ws* edge. We can translate this reasoning in the following equation:

$$\begin{aligned} \forall n, m \in obs_{X_i}, (n < m) \wedge (obs_{X_i}[n] \bmod K = \mu_0) \wedge \\ (obs_{X_i}[m] \bmod K = \mu_1) \implies obs_{X_i}[n] \xrightarrow{ws} obs_{X_i}[m] \quad (3.12) \end{aligned}$$

We are deriving both values from the same buffer to ensure that the indices can be compared. The modulo checks are to make sure that the value $obs_{X_i}[n]$ was written by the W_0 and the value $obs_{X_i}[m]$ was written by W_1 . The last condition $n < m$ compares that we observed the write happened by W_0 before we observed the write happened by W_1 .

Checking for the Reads-From Edge: As for *ws*, we can only have one case for the reads-from edge. If a node $N1$ is connected to a node $N2$ through an *rf* edge, then it implies that node $N1$ is a write node, and node $N2$ is a read node operating on the same variable as $N1$. Hence, the only case we have for the *rf* edge is $W_0[X_i, \mu_0] \xrightarrow{rf} R_1[X_i, r0]$. To check for the *rf* edge, we need to check that the value read by the read node is the same as the value written to by the write node. As in the previous cases, the values written by the write node will be extracted from the observer buffers, and the values read by the read node will come from the read buffer ($r0$). The equation for checking

this edge will be:

$$\forall n \in obs_{X_i}, \forall m \in r0, (obs_{X_i}[n] = r0[m]) \wedge (obs_{X_i}[n] \bmod K = \mu_0) \implies obs_{X_i}[n] \xrightarrow{rf} r0[m] \quad (3.13)$$

As in the previous equations, the modulo check is to make sure that the value is coming from the W_0 write node. The expression $(obs_{X_i}[n] = r0[m])$ checks that the value we read is the same value that was written by the write node.

Checking for the From-Reads Edge: Deriving the fr edge is more complicated than the rest. However, we only have one case to consider. If a node $N1$ is connected to a node $N2$ through an fr edge, then it implies that $N1$ is a read node, and $N2$ is a write node operating on the same variable. Hence, the only case we have is $R_0[X_i, r0] \xrightarrow{fr} W_1[X_i, \mu_0]$. The complexity in this comes because if R_0 is connected to W_1 through fr , then it means the value read by R_0 comes from another node that is connected to W_1 through a ws edge. In other words, we need to check that the value read was written from a write operation that happened prior to W_1 . To solve this, our compiler breaks down an fr edge into $rf + ws$ edges and also adds ‘pseudo’ nodes with ws edges.

Figure 3.7 below explains how the compiler handles this edge using a small example. In this example, we have 3 nodes, and R_2 has an fr edge with W_1 . The nodes marked in green are the ones under consideration for deriving the fr edge. Initially, we just consider the 2 nodes that are directly connected through the edge. Then, because the value read by R_2 needs to be compared to the values written by the write-nodes happening prior to W_1 , we take node W_0 into consideration. Any value that comes from node W_1 but happened at a prior iteration should also have a ws edge with W_1 ’s current iteration. Hence, the compiler creates a pseudo node/edge representing the writes made

by W_1 but in previous iterations and adds it to the set of nodes in consideration. Now, we have to check for ws edges and the rf edges. If one of the rf edges exists with any node that has a ws edge with W_1 , then we will have the fr edge.

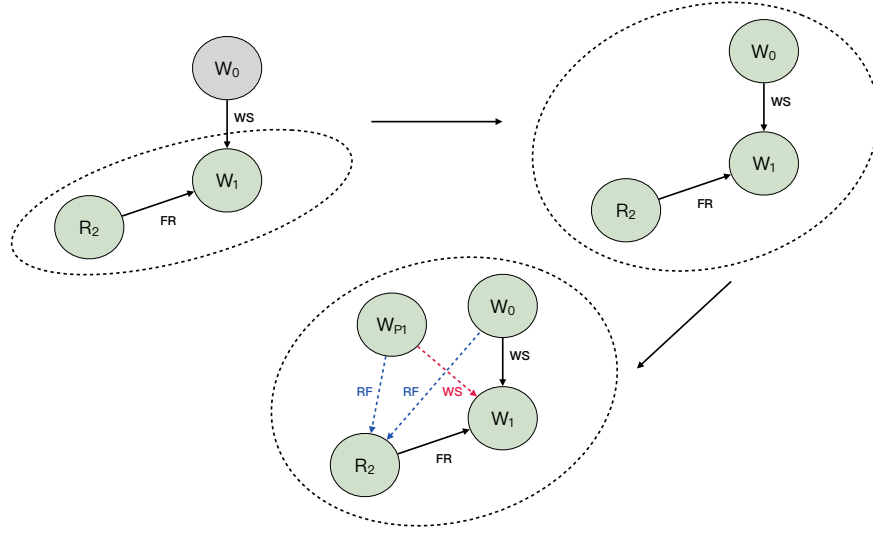


Figure 3.7: Example demonstrating how fr edges are handled by the compiler.

3.5.2.1 Optimizations

With the trace analysis techniques discussed above, if there are n nodes, we will have at least n for loops and at least n conditionals. For the fr edge, we will end up having an additional for loop because of the pseudo node/edge added. Some optimizations can be applied to reduce our search space or the number of loops.

Optimizing the Write Serialization Edge Heuristic: The way we have defined the ws edge, the code representing our equation will have 2 for loops. The pseudo-code has been shown in Algorithm 5.

As discussed in the previous section, we need the condition $n < m$ to be true where n

Algorithm 5 Deriving The WS Edge

```
for  $n = 1$  to total_iterations do
  for  $m = 1$  to total_iterations do
    if  $(\text{obs}_{X_i}[n] \bmod K = \mu_0) \ \& \ (\text{obs}_{X_i}[m] \bmod K = \mu_1) \ \&$ 
       $(n < m)$  then
       $\text{obs}_{X_i}[n] \xrightarrow{ws} \text{obs}_{X_i}[m]$ 
    end if
  end for
end for
```

and m are indices in the same observer buffer. Based upon this condition, we can reduce the search space because checking any m that is less than or equal to n is unnecessary. This allows us to start m from $n + 1$ instead of 1.

Now, if the observer buffer successfully recorded both the values in the ws relation, then it is far more likely that the observations happened closer in time than farther apart. Hence, the values should be spatially close in the observer buffer. This can help us reduce our search space even further because instead of checking all the iterations for which the condition $n < m$ holds, we can check the values within a certain ‘window’ size. This window size determines how spatially far we would like to go in our search. Therefore, a window size of 1 implies that we would be checking the immediate neighbor. A window size of w means we will be checking for a ws edge in values that are up to w iterations apart.

The optimized pseudo-code has been provided in Algorithm 6. Hence, we reduce our complexity from $\theta(T^2)$ where T is the total number of iterations to $\theta(T * W)$ where W is the window size and a constant. This ws edge optimization can be added to the generated code by setting the `ws_pruned` flag to true and changing the `window_size` parameter in the config file.

Algorithm 6 Deriving The WS Edge - Optimized

```
for  $n = 1$  to total_iterations do
  for  $m = n + 1$  to  $m + \text{window\_size}$  do
    if  $(\text{obs}_{X_i}[n] \bmod K = \mu_0) \ \& \ (\text{obs}_{X_i}[m] \bmod K = \mu_1)$  then
       $\text{obs}_{X_i}[n] \xrightarrow{ws} \text{obs}_{X_i}[m]$ 
    end if
  end for
end for
```

Optimizing Heuristic for $R \rightarrow R$ Program-Order Case: We can optimize the $R_0[X_i, r0] \xrightarrow{po} R_1[X_j, r1]$ case similar to how we optimized the *ws* edge heuristic. For checking this case, we simply check whether the index of a read-value from R_0 is lesser or equal to the index of a read-value from R_1 . Hence, instead of iterating over the entire $r0$ and $r1$ buffers, we can reduce our sample space to only iterate over the part of $r1$ such that the indices condition is satisfied. To further reduce the sample space, we can set the window size so we only check for the edge within values that are spatially close. Our pseudo-code can be optimized from the one shown in Algorithm 7 to the one shown in Algorithm 8.

Algorithm 7 Deriving The PO Edge - Case: $R \rightarrow R$

```
for  $n = 1$  to total_iterations do
  for  $m = 1$  to total_iterations do
    if  $n < m$  then
       $r0[n] \xrightarrow{po} r1[m]$ 
    end if
  end for
end for
```

This po optimization can be added to the generated code by setting the `po_pruned` flag to true and changing the `window_size` parameter in the config file.

Algorithm 8 Deriving The PO Edge - Case $R \rightarrow R$ - Optimized

```
for  $n = 1$  to total_iterations do
  for  $m = n + 1$  to  $m + \text{window\_size}$  do
     $r0[n] \xrightarrow{po} r1[m]$ 
  end for
end for
```

Preliminary Check Optimizations: When we have a test with n nodes and edges, then the general structure of the heuristics would be to have for loops for deriving each of the node values from the respective observer buffers or the read buffers and then doing all the checks for the conditions in the innermost for-loop. For example, consider the 2+2 Write litmus test, which is shown in figure 3.8 below. In this test, we have 4 write nodes: nodes a and b execute on 1 thread, and nodes c and d execute on another thread. If we consider optimizing for the ws edge, then Algorithm 9 shows the heuristics for checking weak memory behavior in this test. The 1st for loop is for getting a value that was written by W_0 , the second for loop is for getting the value written by node W_1 , the third for loop is for getting the value written by node W_2 , and the fourth for loop is for getting the value written by node W_3 . The 3rd and 4th for-loops have been optimized because of the ws edge. Once we have the possible candidates, we do the checks to confirm whether the values we saw actually come from the respective nodes (modulo checks from lines 5 to 8). Then the conditionals in lines 9 and 10 check for the two po edges, and the conditionals in lines 11 and 12 check for the ws edge. In the code produced by the compiler, the conditionals in lines 11 and 12 will be omitted because they are already insured by the start and end indices of the last two loops. However, we have included them with this pseudo code for clarity.

The problem with this structure is we check whether the values actually come from the respective write-nodes in the inner-most for loop. Hence, suppose we have a million iterations, and we are on an iteration $i0$. Now, it is possible that once we reach line 5, the mod conditional for W_0 fails, which means the value in $obs_X[i0]$ was not written to by node W_0 . In such cases, we will be wasting computation time because we will have to execute the second for loop for 1M iterations for this invalid value. The way to resolve this is to shift these modulo-checks above and break out of the loop early if we see an invalid value before coming to the edge conditionals. The optimized code has been provided in Algorithm 10.

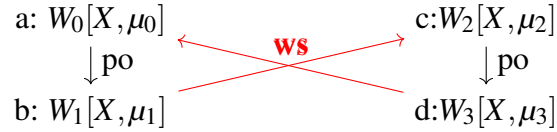


Figure 3.8: 2+2 Write Litmus test graph.

Reducing the Number of Loops for Tests The PerpLE paper [15] proposes heuristics that can be used to reduce the complexity of the trace analysis checks for detecting weak memory behavior to $\theta(n)$. In our initial review, we did not fully understand the reduced heuristics in PerpLE, due to the natural complexity of perpetual tests and their corresponding logs. Upon further reflection and analysis, we derived a similar heuristic that is specifically tuned to how our compiler generates perpetual instance tests, observer buffers, and store values to reduce the trace analysis time complexity to $\theta(n)$ by using the concept of ‘synchrony point’. This involves iterating over the read buffers and observer thread buffers to find an iteration where the threads were synchronized and checking that iteration for weak memory behavior conditions. Upon revisiting the

Algorithm 9 Pseudo code for the 2 + 2 write litmus test heuristics.

```
1: for  $i_0 = 0$  to total_iterations do
2:   for  $i_1 = 0$  to total_iterations do
3:     for  $i_2 = i_1 + 1$  to  $i_2 + \text{window\_size}$  do
4:       for  $i_3 = \max(0, i_0 - 1)$  to  $i_3 - \text{window\_size}$  do
5:         if ( $\text{obs\_X}[i_0] \bmod k = \mu_0$ ) &
6:           ( $\text{obs\_X}[i_1] \bmod K = \mu_1$ ) &
7:           ( $\text{obs\_Y}[i_2] \bmod K = \mu_2$ ) &
8:           ( $\text{obs\_Y}[i_3] \bmod k = \mu_3$ ) &
9:           ( $\text{obs\_X}[i_0] < \text{obs\_Y}[i_1]$ ) &
10:          ( $\text{obs\_Y}[i_2] < \text{obs\_X}[i_3]$ ) &
11:          ( $i_1 < i_2$ ) &
12:          ( $i_3 < i_0$ ) then
13:            WEAK MEM BEHAVIOR FOUND
14:          end if
15:        end for
16:      end for
17:    end for
18:  end for
```

PerpLE paper, we realized that our independently derived heuristics closely aligned with their approach for tests with 2 read instructions (MP, SB, and LB). However, our optimized heuristics for other tests, such as the 2+2 Write, Store, and Read tests, are unique and distinct from the approach in the PerpLE paper.

Idea of synchrony point: In perpetual instance tests, we synchronize the for loops once before they begin, and there are no guarantees that the iterations of the loops will execute in perfect synchrony. For example, we cannot assume that the n th iteration of thread 0 will be synchronized with the n th iteration of thread 1. Thread 1 could run certain iterations ahead or behind thread 0 and vice versa.

If iteration n of thread 0 was synchronized with iteration m of thread 1, it means that the iteration n of thread 0 executed temporally close to iteration m of thread 1. We are more likely to find a weak memory behavior, if it happened, by querying the values

Algorithm 10 Pseudo code for optimized heuristics for the 2 + 2 write litmus test.

```
1: for  $i0 = 0$  to total_iterations do
2:   if  $\text{obs\_X}[i0] \bmod k \neq \mu_0$  then
3:     continue
4:   end if
5:   for  $i1 = 0$  to total_iterations do
6:     if  $\text{obs\_X}[i1] \bmod K \neq \mu_1$  then
7:       continue
8:     end if
9:     for  $i2 = i1 + 1$  to  $i2 + \text{window\_size}$  do
10:      if  $\text{obs\_Y}[i2] \bmod K \neq \mu_2$  then
11:        continue
12:      end if
13:      for  $i3 = \max(0, i0 - 1)$  to  $i3 - \text{window\_size}$  do
14:        if  $\text{obs\_Y}[i3] \bmod k \neq \mu_3$  then
15:          continue
16:        end if
17:        if  $(\text{obs\_X}[i0] < \text{obs\_Y}[i1]) \ \& \ (\text{obs\_Y}[i2] < \text{obs\_X}[i3]) \ \& \ (i1 <$ 
18:           $i2) \ \& \ (i3 < i0)$  then
19:          WEAK MEM BEHAVIOR FOUND
20:        end if
21:      end for
22:    end for
23:  end for
```

at the observer/read buffer indices that correspond to iteration n of thread 0 and iteration m of thread 1. Hence, instead of comparing all the indices of all the buffers with each other, we can find this synchrony point and only check the neighboring indices of this synchrony point because these indices would have executed closer in time, so they will have a higher probability of encoding a cycle. If iteration n of thread 0 happened close/at iteration m of thread 1, then comparing the buffer index that has a value corresponding to iteration n of thread 0 with indices that have values corresponding to any other iteration t such that $t \gg m \vee t \ll m$ is less likely to reveal a weak memory behavior cycle, as those iterations did not execute in close temporal proximity.

Figure 3.9 shows the concept of finding a synchrony point logically. The grey boxes represent when the different iterations of thread 0 and thread 1 execute. We can see that iteration N_0 of thread 0 executes when iteration M_0 of thread 1 executes. We call this point as the ‘synchrony point’! If any reordering or weak memory behavior happened at iteration N_1 , then we are more likely to capture it at M_0 than at any other iteration/index such as M_5 that happened far apart from N_0 , which is why we would just want to check the iteration of the synchrony point or its neighboring indices.

Equation 3.14³ defines the derivation of the synchrony point. If we find a synchrony point, then it implies that we should be able to validate one of the edges in our execution graph at that point. The converse of this should be true as well. Hence, if we are able to validate an edge in our graph at indices n and m , then it implies that the iterations that indices n and m in the buffer correspond to happened temporally close, giving us our synchrony point. The set of edges E in the equation would be the set of diagonal edges: ws , rf , and fr . This is because only these three edges connect nodes across different threads. Validating po edges cannot give us the synchrony point because it connects

³In the equation, ‘buf’ can be either an observer buffer or a read buffer.

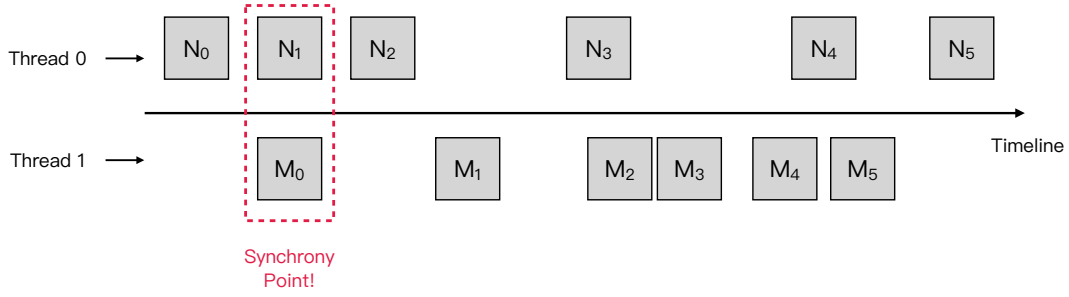


Figure 3.9: Example demonstrating the concept of synchrony point.

nodes within the same thread.

$$\begin{aligned} \exists e \in E, \text{SynchronyPoint}(n, m) &\implies \text{buf}[n] \xrightarrow{e} \text{buf}[m] \Leftrightarrow \\ \exists e \in E, \text{buf}[n] \xrightarrow{e} \text{buf}[m] &\implies \text{SynchronyPoint}(n, m) \quad (3.14) \end{aligned}$$

The synchrony point heuristics for the MP, SB, and LB tests are identical to the PerpLE heuristics, as mentioned before. We comprehensively explain the heuristics for MP, SB, and LB and the unique heuristics developed for other tests, such as 2+2 Write, Store, and Read tests, below.

- **Store Buffer:** SB has an 2 fr edges (execution graph shown in Figure 3.10).

As we have described in the above section, we need to traverse the read buffers $r0$ and $r1$ to infer the values associated with nodes R_1 and R_3 . Suppose, for an iteration i , the value of $r0[i]$ is x . Now, x must have been stored by W_2 in some iteration m . We can get the value of m by $(r0[i] - \mu_2)/K$, where K is the constant we defined in section 3.4. This tells us that iteration i of thread 0 must have been synchronized with iteration m of thread 1. Now, $r0[i]$ should have an rf edge

with the write made by W_2 in iteration m because the value that is read by $r0$ in iteration i was written in iteration m of thread 1. Ideally, we would also want to check our observer buffers to make sure that we saw x in one of the observer buffers for Y . However, R_1 reading the value x is sufficient to prove that this value would have been written by W_2 at some point, and checking for it in the observer buffers becomes redundant.

If i has an rf edge with m , then it should have an fr edge with iteration $m + 1$ because the write in iteration m would have a ws edge to the write made in iteration $m + 1$. This is our synchrony point. Hence, we have derived our first fr edge at this point. Now, we can check the value in $r1[m + 1]$, and if this value has an fr edge with the write made by W_0 in iteration i , then we have a weak memory behavior! To check for this second fr edge, we just need to make sure that $r1[m] < \omega_0^i$ where ω_0^i is the write made by W_0 in iteration i and is equal to $(K * i) + \mu_0$. If this condition is true, it means the value read in $r1[m]$ comes from the write made by W_0 in some iteration prior to i , and this prior iteration will definitely have a ws edge with the write made in iteration i , confirming our second fr edge!

- **Message Passing:** MP has an fr edge and an rf edge (execution graph shown in Figure 3.11). We can infer these edges as we did for SB, and the po edges become trivially implied because of the diagonal edges. As we did in SB, we need to start by traversing the read buffer for R_2 , $r0$. Suppose the value in $r0$ for an iteration i is x . Because R_2 reads from Y , this value would have come from the node W_1 in thread 0. We can get the iteration in which x was written by $(r0[i] - \mu_1)/K$, where K is the constant we defined in section 3.4. We denote this iteration as m .

By this, we can infer that iteration m of thread 0 was synchronized with iteration i of thread 1 (point of synchrony), and we have confirmed the rf edge.

Now, because R_3 executes in the same thread as R_2 , for there to be a weak memory behavior, the value read by R_3 in iteration i must have an fr edge with the write made by W_0 in iteration m of thread 0. To confirm this fr edge, we need to check if $r1[i] < \omega_0^m$ where ω_0^m is the value written by W_0 in iteration m of thread 0. This comparison implies that $r1[i] = \omega_0^{<m}$ and $\omega_0^{<m} \xrightarrow{ws} \omega_0^m$ will always hold, confirming the fr edge.

- **Load Buffer:** LB has 2 rf edges (execution graph shown in Figure 3.12). For inferring them, we can use the same methodology that we used for SB and MP. We don't need to infer the po edges because they will be trivially implied. We need to traverse the read buffer $r0$ to infer the first rf edge. As in the previous cases, suppose the value in buffer $r0$ for iteration i is x . This value would have come from iteration m of W_3 where $m = (x - \mu_3)/K$. This means there is an rf edge between $r0[i]$ and the write made by W_3 in iteration m , and we have found the point of synchrony.

Now, to check whether we have a weak memory behavior, we need to check $r1[m]$ and ensure that it has an rf edge with ω_1^i which is the write made by W_1 in iteration i of thread 0. To ensure this relation and confirm the second fr edge, we need to check that $r1[m] = (K * i) + \mu_1 = \omega_1^i$, which simply means that the value that was read by R_2 in iteration m was written by W_1 in iteration i .

- **2+2 Write:** The 2 + 2 Write litmus test has 2 ws edges that connect nodes in thread 0 to nodes in thread 1 (execution graph shown in Figure 3.8). As in the

previous three litmus tests, we do not have read buffers to infer this edge. Instead, we use the observer buffers for X and Y . An index n in an observer buffer will have a ws edge with every index m such that $m > n$ because the value at index n was observed before the value at index m . For this reason we specify a window size such that we only go up to a certain t iterations for inferring the ws edge. This implies, that instead of formulating a ws edge from the index n of a buffer with every index $> n$, we only consider the ws edges from index n to indices in the range $[n + 1, n + t]$ such that the modulo conditions are satisfied (value at index n comes from the parent node of the ws edge, and the value at the index $> n$ comes from the child node of the ws edge).

Suppose we iterate over the observer buffer of X and take the ws edge from index n to index n' where $n' \in [n + 1, n + t]$ such that the modulo conditions are satisfied, then the value at index n must have been written in some iteration t of thread 0, and the value at index n' must have been written in some iteration t' of thread 1. This is our synchrony point because it means iteration t of thread 0 must have been executed temporally close to the iteration t' of thread 1. Now, we can investigate the indices n and n' in the observer buffer of Y that is from the same observer thread as the observer buffer of X and check whether we have the other po and ws edges. If we do, then we have a weak memory behavior. In these heuristics, we cannot use the observer buffers from two different observer threads together because then we will have to consider an additional point of synchrony between both observer threads.

- **Store and Read:** The Store and Read tests are similar, so we discuss them together. In the Store test, we have 1 ws edge and 1 rf edge (execution graph

shown in Figure 3.13). For an index t of the read buffer $r0$ where $x = r0[t]$, we can calculate the iteration in which the value x must have been written in thread 0. Suppose that iteration in thread 0 is t' where $t' = (x - \mu_1)/K$. Now, we know that iteration t' of thread 0 must have been synchronized with iteration t of thread 1. This is our first synchrony point. However, we still need to find a synchrony point between the observer thread and thread 1. In the observer buffer for X , we can find an index t'' such that the value at t'' has a *po* edge with x . This would be our second synchrony point which will give us the information about which iteration of the observer thread was synchronized with iteration t' of thread 1 (t'' in our example). Now, if we can find a value in the same observer buffer such that it has a *ws* edge with $buf[t'']$, and its value is $\leq (K * t') + \mu_0$, then we have inferred the cycle, and we have a weak memory behavior. We can use the same logic for the Read litmus test to find both the synchrony points. The only difference now would be that the second synchrony point would have to be found using the *fr* edge.

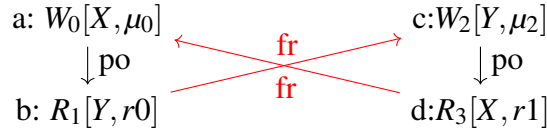


Figure 3.10: Store Buffer litmus test graph.

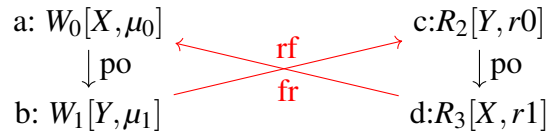


Figure 3.11: Message Passing litmus test graph.

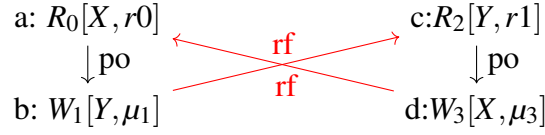


Figure 3.12: Load Buffer litmus test graph.

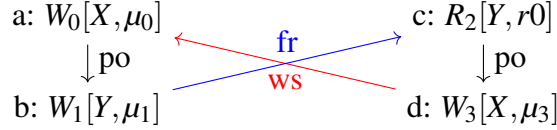


Figure 3.13: Store litmus test graph.

3.6 Optimizations Injected to Observe More Weak Behavior

The compiler adds three kinds of optimizations in the litmus tests to increase the weak memory behavior throughput (number of weak behaviors observed per second). The impact of these optimizations has been shown in our results chapter.

Memory Stress: It has been observed in parallel works that adding memory stress to the system, especially on GPUs, influences the number of weak behaviors observed [21]. In our testing, we implement memory stress on both the CPU and the GPU. The compiler produces code so that the number of CPU stress threads can be specified while running the test file as a command line argument. On the GPU side, any thread that is not involved in the main litmus test or making observations is assigned to implement memory stress. The stress threads do a memory-intensive operation, such as blurring, on a shared memory array. The pseudo codes for our CPU and GPU stress functions have been provided in Algorithms 11 and 12.

Algorithm 11 CUDA function for GPU memory stress

```
function GPUMEMORYSTRESS(volatile int* stress_array, int tid)
  for  $i = 0$  to  $x$  do
     $x \leftarrow stress\_array[tid + 1]$ 
     $stress\_array[tid] \leftarrow 90$ 
  end for
  return
end function
```

Algorithm 12 CPU memory stress function

```
function CPUMEMORYSTRESS(int tid, volatile int* array, volatile
int* stop_thread, int array_size, int num_threads)
  while  $stop\_thread[tid] \neq 1$  do
    for  $i = tid + 1$  to  $array\_size - 1$  do
       $array[i] \leftarrow 1 + array[i - 1] + array[i + 1]$ 
       $i += num\_threads$ 
    end for
  end while
end function
```

The stress threads on the CPU are launched before the iterations begin. Once all the main tests finish, the stress threads are signaled using the `stop_thread` array to exit. On the GPU, we don't have a `stop_thread` array; instead, we implement the stress for an estimated number of iterations x such that the stressing goes for as long as the main test.

Random Thread ID: We observed that the number of weak behaviors increased if we randomly selected the thread ID for performing the main test. In this optimization, we select a random thread ID on the CPU and a random thread ID on the GPU to perform the main test. In the case of single instance tests, this random selection is made for

every iteration. For perpetual instance tests, things become a little challenging on the GPU. Because of this, we have not yet implemented this specific optimization for the perpetual instance tests.

Optimized Stores: In this optimization, we pre-compute and store the memory address of the index in the shared array to which a store/write has to be made in a pointer before the iteration/main test begins. This optimization helps us avoid recalculating the memory address in every iteration or while executing the nodes because it is a costly operation. In our experiments, we noticed running perpetual tests without this optimization takes a huge amount of time to finish for a million iterations, because of which we added this optimization in all perpetual tests by default. Algorithms 13 and 14 below show the difference in code before and after this optimization for one of the SB test threads.

Algorithm 13 SB test - without optimization

CPU Thread:

```

    SYNCHRONIZE
    for  $i = 1$  to total_iterations do
         $com[index\_x] = 1$ 
    end for

```

Algorithm 14 SB test - with optimization

CPU Thread:

```

    int* addr_X = com + index_x
    SYNCHRONIZE
    for  $i = 1$  to total_iterations do
         $*addr\_X = 1$ 
    end for

```

Chapter 4 demonstrates the throughput improvement and speedup we observed us-

ing this optimization for single-instance tests.

Chapter 4

Experiments and Results

4.1 Devices Tested

We tested three CPU-GPU integrated devices: (1) AMD Ryzen 7 5700G with Radeon Graphics, (2) Intel i7-12700K with the UHD Graphics 770 integrated chip, and (3) NVIDIA Jetson Orin Nano 8GB. With the first two devices, we faced significant challenges in allocating shared memory using Vulkan. In our single instance runs, we could only see sequential behavior and no interleaved executions. Upon further investigation, we found that we consistently only observed the initialized values in every run and iteration. We hypothesized that this might be because an iteration of a single instance test finishes very quickly and probably does not give enough opportunity for an interleaving or a weak memory behavior to happen. Hence, we shifted our focus to perpetual instance testing on these devices, which should have allowed for more interleaving in the instructions. We conducted our initial experiments with 100,000 iterations using the message passing litmus test. However, we observed surprising behavior. In most of the runs, we observed 0 weak behaviors, and in some rare runs, we

observed 100% weak behaviors or weak behaviors in all 100,000 iterations. We found that this was happening because the read nodes loaded the same values in all the iterations. Most of the time, this value was the initialized value, which is 0. Sometimes, the consistent value read in every iteration was different from 0 and corresponded to a weak memory behavior, which is why, in some runs, we recorded 100% weak behaviors. We tried removing the synchronization from before the loop, but it did not help us see any differences in the behavior. We suspected that the communication between the CPU and GPU was not working as we had intended, and to confirm this speculation, we did another set of experiments.

Instead of having the barrier synchronization before the for-loop, we put it inside the loop for every iteration. Now, we incremented a single shared memory location in the GPU thread and the CPU thread, and, in our barrier synchronization, we spun till the value of the memory location was 2 + its previous value, confirming that the CPU and GPU can see the updates made by each other. Algorithm 15 shows the pseudo-code for our experiment. We observed that the program was getting stuck and never finishing. We then added a print statement in the CPU thread after the while loop to check whether we were getting stuck in the first iteration or if we were able to go up to a certain number of them. We realized that after a few seconds, we could get up to 2- 3 iterations ahead in some runs on both devices, but the loop was getting stuck after that, and the CPU could not observe the increment made by the GPU. This observation also provided an explanation for why the program was not getting stuck in the initial experiments when we had a single synchronization barrier before the for loops. The CPU and GPU can read updated values until certain iterations, but after that, the communication fails. These experiments confirmed our analysis that the CPU and GPU communication was

not working. Because of these difficulties, we shifted our focus to testing the NVIDIA device, on which our preliminary runs were successful, and we observed interleaved and weak behaviors. In the future, we aim to test the Intel and AMD devices further and investigate why the communication is not working. The sections below describe the experiments we performed on the NVIDIA device and our observations.

Algorithm 15 CPU-GPU communication testing experiment

CPU Thread:

```

for  $i = 1$  to total_iterations do
   $x \leftarrow x + 1$ 
  while  $x < (i * 2)$  do
    end while
  end for

```

GPU Thread:

```

for  $i = 1$  to total_iterations do
   $x \leftarrow x + 1$ 
  while  $x < (i * 2)$  do
    end while
  end for

```

4.2 Experiments Performed

The Tables 4.1 and 4.2 define the experiments we did using the single instance approach and the perpetual instance approach. All these experiments were performed on an NVIDIA Jetson Orion Nano with a 1024-core NVIDIA Ampere architecture GPU and an integrated ARM Cortex-A78AE v8.2 64-bit CPU. The first column in the tables specifies all the different litmus tests we did. The second column specifies the different GPU fences we used between the two instructions/nodes on the GPU thread to eliminate re-orderings, and the third column specifies the CPU fence we used between

the two instructions/nodes on the CPU thread to eliminate reordering. These fences were not used in the observer threads and were only used to eliminate re-orderings in the main test threads. The CPU | GPU test direction means that the first two nodes were executed on the CPU, and the last two nodes were executed on the GPU. Similarly, the other direction means the last two nodes in the litmus tests were executed on the CPU and the first two nodes on the GPU.

Test Names	GPU Fences	CPU Fences	Test Direction	Optimiz- -ations
Message Passing Store Buffer Load Buffer Read Store 2+2 Write	blockfence systemfence threadfence	atomic_thread_fence (memory_order_seq_cst)	CPU GPU GPU CPU	Memory Stress Random TID Optimized Stores/Loads

Table 4.1: Experiments performed with single-instance approach. In the table, the GPU fence ‘blockfence’ refers to the fence `__threadfence_block()`. The fence ‘systemfence’ refers to the fence `__threadfence_system()`, and the fence ‘threadfence’ refers to the fence `__threadfence()`.

Test Names	GPU Fences	CPU Fences	Test Direction	Optimiz- -ations
Message Passing Store Buffer Load buffer 2+2 Write Store	blockfence systemfence threadfence	atomic_thread_fence (memory_order_seq_cst)	CPU GPU GPU CPU	Memory Stress

Table 4.2: Experiments performed with perpetual-instance approach. In the table, the GPU fence ‘blockfence’ refers to the fence `__threadfence_block()`. The fence ‘systemfence’ refers to the fence `__threadfence_system()`, and the fence ‘threadfence’ refers to the fence `__threadfence()`.

For the single instance approach, we generated 24 configurations for each test (using different combinations of direction, GPU fence, and CPU fence specified in the table). Each of these configurations was tested with the three different optimizations given in the table, giving us a total of 72 configurations for each litmus test. Each of these configurations was run 500 times (average) with around 300 - 500 iterations in each run, using different fuzzed parameters such as the difference in index of X and Y in the shared array, number of CPU stress threads, size of the shared array, etc.

All of our perpetual instance tests already had the ‘optimized stores’ optimization because computing about 100,000 to a million iterations without it had a huge latency that was overpowering the speedup we got through the perpetual instance approach. We describe below the perpetual test configurations generated for different litmus tests :

- **LB, SB, and MP:** We generated 12 configurations for the LB, SB, and MP litmus tests using different combinations of directions with fences on the CPU and GPU side. Each of these configurations was tested with and without the memory stress optimization, giving us a total of 24 configurations for each litmus test. We used the most optimized heuristics (single for loop) for these tests, which omits the need for observer threads because we do not have to check the buffers to infer the po and ws edges. However, the observer threads were still included in our test files (1 CPU and 1 GPU observer thread) as they acted like stress threads. We ran each of the non-stress tests an average of 200 times with a million iterations in each run. For the stress tests, we did slightly more runs because we had some additional parameters to fuzz, such as the number of stress threads on the CPU and the size of the stress array.
- **2 + 2 Write:** For 2+2 Write, we used our heuristics with the ws edge pruned

(hence, there were 2 logical for loops). In these tests, we generated configurations with differing window sizes and observer threads to observe the change in weak memory throughput. All the configurations were run with and without stress. As in the case for LB, SB, and MP tests, we ran the configurations about 200 times (non-stress), and the stress configurations were run slightly more times, with 10,000 iterations in each run. The iterations are lower as compared to LB, SB, and MP because we make multiple calls to the heuristic function that has a $\theta(n^2)$ time complexity. In LB, SB, and MP, we make a single call to the heuristic function that has $\theta(n)$ complexity. For 2+2 Write, we call the function with all the combinations of the X and Y observer buffers. Hence, we have 4 calls to a $\theta(n^2)$ heuristic function when we have 1 observer thread on the CPU and 1 observer thread on the GPU. This is because we will have 2 observer buffers for both X and Y , and we can make 4 combinations with them. When we have 2 threads on the CPU and GPU, then the number of calls to the heuristic function increases to 16 because we have 4 observer buffers for each variable, and we can make 16 combinations out of them.

- **Store:** For the Store test, we used our heuristics with the *ws* edge pruned. Our configurations had 1 CPU observer thread, 1 GPU observer thread, and the window size parameter set to 1. We ran the configurations with 1000 iterations about 200 times. The stress configurations were run slightly more times because we had 2 additional parameters to fuzz as described previously. Our experiments for the Store litmus test had 10x fewer iterations than for the 2+2 Write tests because of the post-processing work (trace analysis heuristics) complexity. Our post-processing complexity is $\theta(n^3)$ for the Store tests because we have 3 log-

ical for loops. As we pruned with *ws* edge, we were able to reduce the time complexity from $\theta(n^4)$ to $\theta(n^3)$. Our post-processing complexity is also heavily dependent upon the observer buffers. If the observer buffers read unique values, the post-processing takes longer. We did a set of experiments to confirm this speculation.

We ran the Store test with 2 CPU and GPU observer threads, without any CPU and GPU fence, and with window size 1 for 1000 and 3000 iterations. Apart from the iterations, all the other fuzzed parameters, such as the array size or the indexes of X and Y, were kept the same for both configurations in every run. Figure 4.1 presents a plot illustrating the post-processing time observed for different runs with 1000 iterations, and Figure 4.2 shows a plot illustrating the post-processing time observed for the runs with 3000 iterations. We can see that the points have a prominent variance even though they were run with the same fuzzed parameters. In Figure 4.2, there is a run for which the post-processing took 0.01 seconds to execute and then another run for which the post-processing took close to 200 seconds to execute. This variation is because of our observer buffers. In the run that took 0.01 seconds, the observer threads probably could not record any interesting data, but the observer buffer data was perhaps unique and interleaved in the run that took close to 200 seconds. For the runs with 3000 iterations, we observed a mean of 42.02 seconds with a standard deviation of 54.22 seconds. For the runs with 1000 iterations, we observed a mean of 4.405 seconds with a standard deviation of 5.59 seconds. For 1000 iterations with 1 CPU and GPU observer thread, our mean and standard deviation were almost the same as those for runs with 1000 iterations that had 2 CPU and GPU observer threads. In our

2+2 Write litmus test experiments, we noticed that adding more observer threads did not significantly change weak behaviors. This is why we decided to use 1 CPU and GPU observer thread and 1000 iterations in our tests.

CPU Fence	GPU Fence	Stress	Window Size	Direction	Throughput
seq_mem	blockfence	FALSE	1	GPU CPU	21.7181
seq_mem	NF	FALSE	1	GPU CPU	22.2384
NF	NF	FALSE	1	GPU CPU	200.6791
seq_mem	NF	TRUE	1	GPU CPU	2.4012

Table 4.3: Store litmus test perpetual instance significant results.

Hence, our trace analysis heuristics sometimes mask the benefits of the perpetual instance approach, and doing fewer iterations is not a good solution as it would not give the observer threads enough opportunity to record interesting data. For this reason, we could not see much weak behavior with the Store litmus test. We were able to see a throughput of > 0 for only 4 configurations shown in Table 4.3. For similar reasons, we were not able to observe any significant data for the Read litmus test.

Our observer threads' inconsistency is also why we could not achieve any significant results or see weak behaviors with the synchrony point optimization for the Store/Read and 2+2 Write tests. For these tests, our synchrony point optimization relies upon recording specific patterns in the observer buffers. If the data recorded by the observer buffers is skewed to just one node (it consistently only records values stored by one node) or has repeated data in chunks in the buffers, then the synchrony point methodology does not work as expected. In our pilot runs with the synchrony point optimization, we observed that the observer buffers

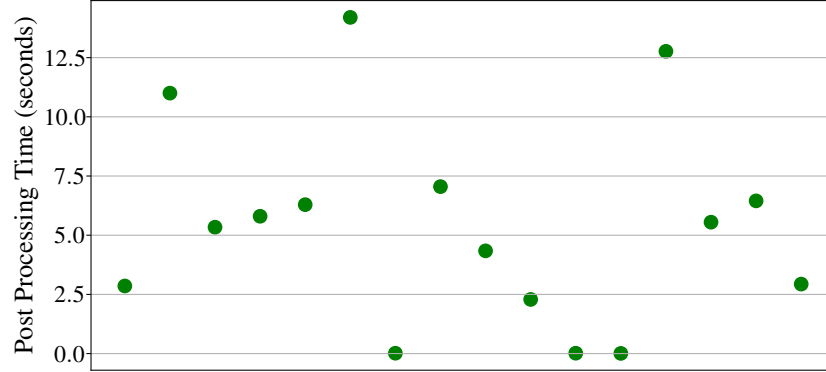


Figure 4.1: Post processing runtime (seconds) for Store test runs with 1000 iterations.

often have repeated data. For example, from iterations n to $n+t$, the buffer would have the same consistent value from one of the nodes, and then from t to $t+t'$, it would have the same consistent value from another node or the same node but from a different iteration. However, we are currently working on incorporating some relaxations in the synchrony point methodology so that our conditionals for checking weak memory behavior do not heavily rely on checking for very specific patterns.

The next 2 sections provide a summary and an analysis of the results from our experiments contrasting our original hypothesis with our observations.

4.3 Single Instance Test Results

With the data obtained from the single instance test experiments, we constructed 2 sets of graphs ¹:

¹In these graphs, we did not plot the configurations for which we observed no weak behaviors. Hence, the graphs do not show `systemfence` and `threadfence` test configurations as we did not observe any

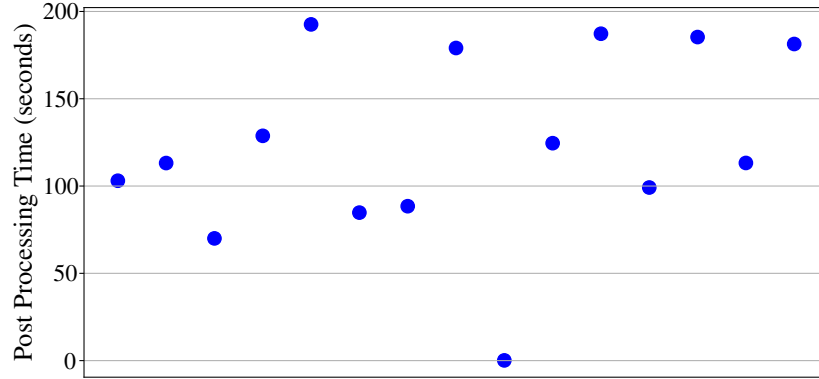


Figure 4.2: Post processing runtime (seconds) for Store test runs with 3000 iterations.

- The first set comprising of Figures 4.3, 4.4, 4.5, 4.6, 4.7, and 4.8 illustrates the percentage of weak memory behavior obtained for different litmus tests using the three optimizations discussed in section 3.6. Weak behavior percentage is calculated using the formula:

$$\text{Weak Behavior \%} = \frac{\text{Total Weak behavior}}{\text{Total iterations}} * 100 \quad (4.1)$$

On the y-axis of these graphs, we have different test configurations, denoted by codes such as $A + B + C$; for example, `block+NF+CG`. The three parameters, A, B, and C, represent different aspects of the configuration:

- **First parameter:** The first parameter represents the fence used in between instructions on the GPU thread. It can have the following four values:
 - * NF: This means that no fences were used in between the instructions on the GPU thread.

weak behaviors for them.

- * block: If the first parameter is ‘block’, then it means that a `__thread fence_block();` was used in between the two instructions on the GPU thread.
 - * system: If the first parameter is ‘system’, then it implies that a `__thread fence_system();` was used in between the two instructions/nodes on the GPU thread.
 - * thread: This value indicates that a `__threadfence();` was used between the two instructions on the GPU thread.
- **Second parameter:** The second parameter represents the fence used on the CPU thread. It can have 2 values:
- * NF: This indicates that no fences were used in between the instructions on the CPU thread.
 - * seq_mem: This implies that an atomic thread fence, `atomic_thread_fence(memory_order_seq_cst),` was used in between the two instructions on the CPU thread.
- * **Third parameter:** The third parameter specifies the direction. It can have 2 values:
- CG: This value implies that the first thread was executed on the CPU, and the second thread was executed on the GPU.
 - GC: This value implies that the first thread was executed on the GPU, and the second thread was executed on the CPU.

The blue bars in the graphs show the percentage of weak memory behavior obtained using just the memory stress optimization, the red bars show the percentage of weak behavior obtained using the memory stress optimization and the

random TID optimization, and the yellow bars show the percentage of weak behavior obtained using all three optimizations.

- The second set comprising of Figures 4.9, 4.10, 4.11, 4.12, 4.13, and 4.14 illustrate the impact of the direction of test on the weak behavior percentage. On the Y-axis, we have similar configuration codes as in the previous set of graphs. The blue bars show the CPU | GPU direction, and the red bars show the GPU | CPU direction.

For completion, we have included the raw data obtained for the 6 litmus tests using the single instance approach in Tables A.1, A.2, A.3, A.4, A.5, and A.6. The direction column specifies which nodes are getting computed on the CPU and the GPU. The direction ‘CPU | GPU’ signifies that the first two nodes were executed on the CPU, and the last 2 nodes were executed on the GPU. Similarly, ‘GPU | CPU’ means that thread 0 was executed on the GPU, and thread 1 was executed on the CPU. The tables also list the CPU and GPU fences we used. The values in these columns are the same as the ones we have described above for the second and third parameters of the test configuration codes. The fourth column shows the percentage of weak memory behavior obtained just using memory stress, the next column shows the percentage obtained with random TID optimization combined with memory stress optimization, and the last column shows the percentage when all three optimizations are applied. For threadfence, some of the entries are ‘-’ or nil because we only obtained the results using the highest optimization.

4.3.1 Analysis

We can observe the following trends using the data that can help us understand what might be happening internally.

- We see weak behaviors from every litmus test. This implies that shared memory on the NVIDIA Jetson Orin Nano allows for all four reorderings (Store $\rightarrow$ Store, Load $\rightarrow$ Load, Store $\rightarrow$ Load, and Load $\rightarrow$ Store). Analyzing which litmus tests gave us the best results or more weak behaviors can give us insights into which reorderings are the most common and which are the least.
- We momentarily set aside the fine-grained test configurations, such as the direction and the fences, and take the average of the weak behavior percentage of the different test configurations for the litmus tests. This gives us an overall average weak behavior percentage observed for every litmus test. The average percentages² are shown in Table 4.4. We observe that the Read litmus test gives us the highest average percentage, followed by the 2+2 Write test, Store test, Store Buffer test, Message Passing test, and finally, the Load Buffer litmus test, which gives us the lowest average. The Read litmus test has different sequences on both threads. In thread 0, it has a Store $\rightarrow$ Store sequence; in thread 1, it has a Store $\rightarrow$ Load sequence. Hence, this indicates that the heterogeneous system might be most prone to Store $\rightarrow$ Load and Store $\rightarrow$ Store reorderings. This also explains why the 2+2 Write test gives us the second highest average because it has a Store $\rightarrow$ Store sequence in both threads. The Load Buffer litmus test has the lowest overall average, indicating that the system might be least prone to Load $\rightarrow$ Store reorderings. The Load $\rightarrow$ Store reordering is present in the Store test as well, but it still gives us the third-highest weak behavior percentage. Our hypothesis is that the Store $\rightarrow$ Store sequence majorly contributes to the weak behavior percentage in the Store test. We will revisit these hypotheses in the perpetual litmus test

²We calculated the average percentage using the tests that had all three optimizations.

results analysis.

Litmus Test	Average Weak Behavior Percentage
Load Buffer	0.8761%
Message Passing	2.3279%
Store Buffer	2.5006%
Store	2.5007%
2 + 2 Write	3.0822%
Read	3.88%

Table 4.4: Average weak behaviors seen using different fence-direction configurations for the litmus tests

- In every litmus test, using a `systemfence`³ and `threadfence`⁴ eliminates all weak behaviors, even in configurations with no fences on the CPU thread. However, we still see weak behaviors when we have a `blockfence`⁵ between instructions on the GPU thread. This observation can help us understand how these GPU fences behave. According to the CUDA specification, a `blockfence` is used for flushing the writes so they are visible across all threads of the block that has the thread that issued the write, a `threadfence` is used for flushing the writes so that it is visible across all threads on the GPU, and a `systemfence` is used for flushing the writes so that they are visible to the host and the device [17]. Hence, the scope of a `blockfence` is limited to a block, the scope of a `threadfence` covers the entire GPU, and the scope of a `systemfence` extends to both the CPU and the GPU.

We hypothesize that when we use a `blockfence`, the writes might not be pushed or updated through the main memory. Instead, they might just be flushed to the

³The term ‘systemfence’ refers to using `__threadfence_system()`

⁴The term ‘threadfence’ refers to using `__threadfence()`

⁵The term ‘blockfence’ refers to using `__threadfence_block()`

block's local cache so the threads in that block can see the updated values. This could be the reason why we still see weak behaviors with a `blockfence`. When we have a `threadfence` or a `systemfence`, the writes are flushed to the main memory so they are visible to the entire GPU. However, there is no guarantee that the CPU will see these updates with just a `threadfence`. Even though we do not see behaviors with a `threadfence`, we hypothesize that the weak behaviors can still exist, and we are just not able to observe them using single instance tests. Through our perpetual instance tests, we show that we can still see weak behaviors using a `threadfence`, and they are only truly eliminated when we use a `systemfence` that enables the updates to propagate to the CPU.

An interesting observation in the graph is that sometimes, using a `blockfence` gives us a slightly higher percentage of weak behaviors than without in some tests such as Read, LB, and MP. Typically, this is counterintuitive because a fence should not cause more weak behaviors to happen. To analyze this, we did a deeper statistical analysis. Read and MP had the most significant difference in the percentage of weak behaviors between configuration with and without the `blockfence`. The difference in the percentage of weak behaviors between the configuration with a `blockfence` on the GPU with no fence on the CPU and the configuration with no CPU and GPU fences in the case of the Read test was 2.7637%. Similarly, this difference in the case of MP was 1.528%. In LB, this difference was less than a percent. As the difference was more significant in Read and MP, we ran these configurations of MP and Read again, where we obtained 8 samples for 10 fuzzed parameters so we could perform the T-test. The t-value was positive for some fuzzed parameters and negative for others, indicating that

sometimes the configuration with the `blockfence` gives us better results, and other times having no CPU and GPU fences gives us better results. However, in all our experiments, we could not get a p-value lesser than 0.05, confirming that even though there is a difference between the percentage of weak behaviors observed, this difference is not statistically significant.

- If we analyze the second set of graphs in Figures 4.9, 4.10, 4.11, 4.12, 4.13, and 4.14, we can observe that one of the directions give us higher weak memory percentage in some litmus tests. This difference is most prominent in the Read test, where the average difference in the weak behavior percentage for different configurations is 5.59%, followed by 2+2 Write with an average difference of 3.57%. The Load Buffer, Store, and Store Buffer tests have an average difference between 1 - 1.5%, and the Message Passing test has the lowest average difference of 0.5%. Hence, we only see a statistically significant difference with the Read and the 2+2 Write litmus tests, which have an average difference of above 3%. The Read litmus test is not symmetrical, and because we see a higher percentage of weak behavior when we have the Store $\rightarrow$ Store sequence on the GPU thread, it could indicate that the GPU might be more sensitive to this reordering and the CPU might be more sensitive to the Store $\rightarrow$ Load reordering. The 2+2 Write test is more interesting because the test is symmetrical. Therefore, there should not have been a significant difference regardless of which thread executed on the CPU or the GPU because both ultimately have a Store $\rightarrow$ Store sequence. We believe that these results still do not indicate false positives because we have validated our testing strategy on CPU-only systems that have a well-defined memory model, and the weak behaviors are eliminated when we use the strongest fence. There

are a number of fine-grained factors, including the fuzzed parameters, that could play a role in this deviation, and we aim to investigate this as part of our future work.

- On average, the percentage of weak behaviors improves when we use all three optimizations, which aligns with our hypothesis. In fact, in some cases, such as in Store, we are only able to see considerable amount of weak behaviors when we use all three optimizations. However, message passing is a small outlier because sometimes, we get better results by just using memory stress and random TID optimization. One possibility is that the tests with the second optimization simply got better randomized fuzzed parameters that increased the percentage of weak memory behavior. We tried to infer if certain fuzzed parameters give us more weak behaviors. For example, if the differences in X and Y indices on the shared array impact the amount of weak behavior we see. However, the data did not reveal any specific trends.

4.4 Perpetual Instance Test Results

We plot 2 graphs for the 2+2 Write litmus test results ⁶. The first graph shown in Figure 4.16 depicts the impact of memory stress on the weak memory behavior throughput of different test configurations, and the second graph shown in Figure 4.17 illustrates the impact of window size on the throughput. The throughput gives us information about the number of weak memory behaviors observed per second and is

⁶We did not plot configurations for which we observed a weak memory behavior throughput of 0.

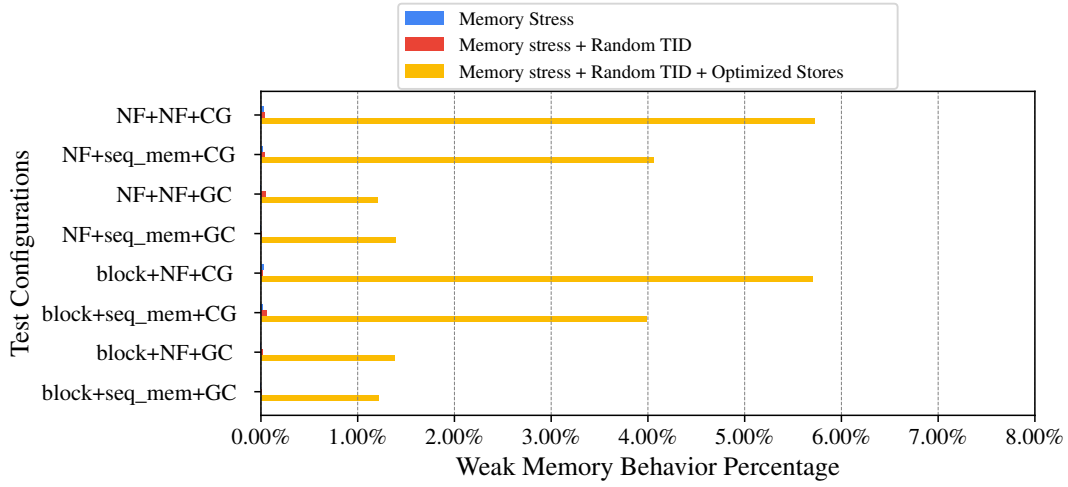


Figure 4.3: Weak memory behavior percentage observed with different optimizations for 2 + 2 Write single instance test configurations.

calculated using the following formula:

$$\text{Weak Behavior Throughput} = \frac{\text{Total Weak Behavior}}{\text{Total Time}} \quad (4.2)$$

On the y-axis, we have similar codes as we did in the case of the single instance test graphs for representing the different configurations. The number after `co` represents the number of CPU observer threads; the number after `go` represents the number of GPU observer threads; the number after `w` represents the window size, and finally, the number after `d` represents the direction (0 = CPU | GPU and 1 = GPU | CPU). For example, `colgo1_w5_d0` means that there was 1 CPU observer thread, 1 GPU observer thread, the window size for the heuristics was 5, and thread 0 was executed on the CPU, and thread 1 on the GPU.

For the other tests, we plot a set of graphs ⁷ (shown in Figures 4.18, 4.19, and

⁷We did not plot configurations for which we observed a weak memory behavior throughput of 0.

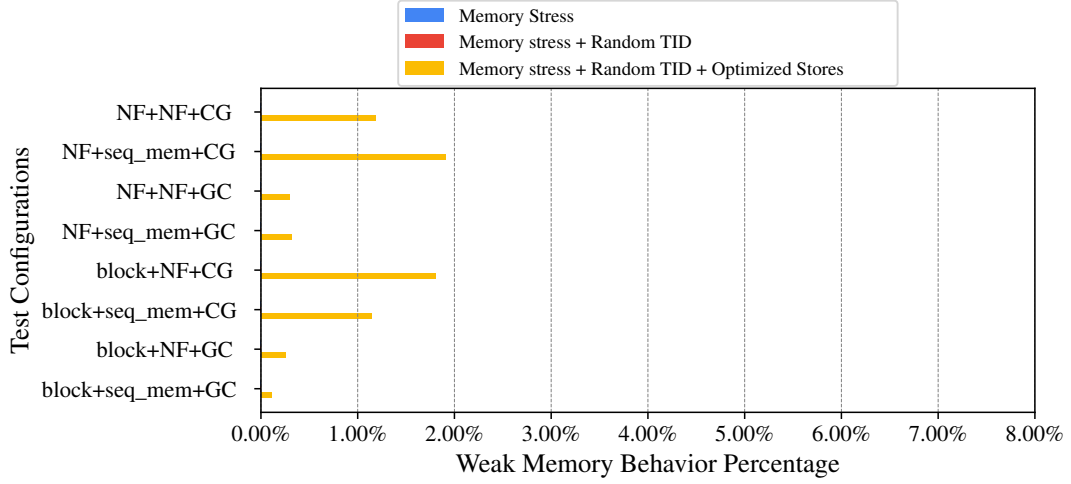


Figure 4.4: Weak memory behavior percentage observed with different optimizations for Load Buffer single instance test configurations.

4.20) to show the difference in weak memory behavior throughput for different configurations with and without memory stress. The y-axis has similar codes to represent the different test configurations. The first parameter in the code represents the type of CPU fence (NF, seq_mem), the second parameter represents the GPU fence (NF, block, system, thread), and the third parameter represents the direction (CG and GC). The configurations on the top have the CPU-GPU direction, and the configurations on the bottom have the GPU-CPU direction in the plots.

For completion, we included the raw data obtained from the tests in tables A.7, A.8, A.9, and A.10. Unlike the tables for the single instance tests, we show the weak memory behavior throughput instead of the weak behavior percentage because this is the parameter that we wanted to optimize through this approach.

In tables A.8, A.9, and A.10, the first two columns show the CPU fence and the GPU fence used. The third column shows whether memory stress was used or not. The

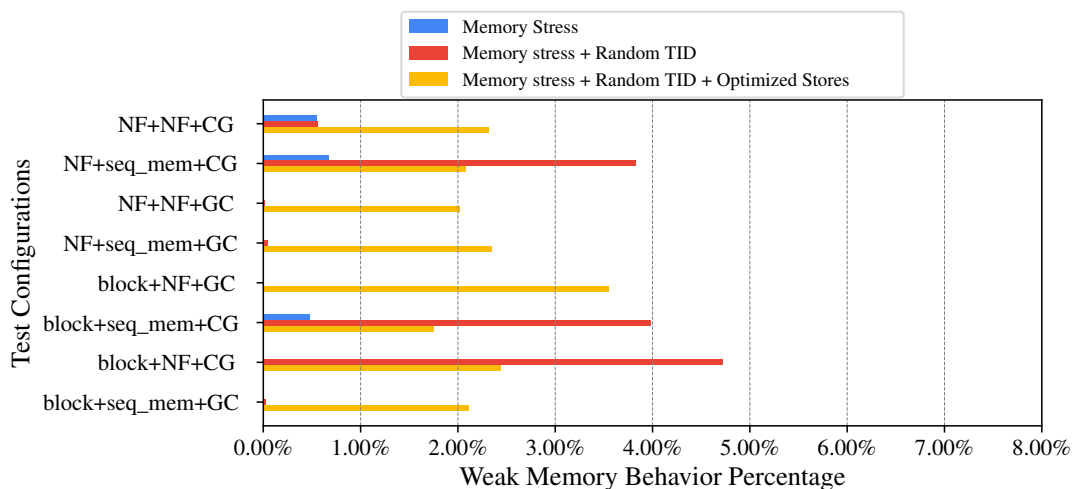


Figure 4.5: Weak memory behavior percentage observed with different optimizations for Message Passing single instance test configurations.

fourth column shows the direction of the test. As in the single instance tests, the CPU | GPU direction means thread 0 was executed on the CPU, and thread 1 was executed on the GPU. Then the last column shows the throughput, which represents the number of weak behaviors observed per second.

The table for the 2+2 write test looks slightly different. This is because we wanted to highlight the functioning of the observer threads through the 2+2 write tests instead of highlighting the differences in the amount of throughput with different fence configurations compared to the single instance tests, which is highlighted well in the LB, SB, and MP perpetual instance results. This is why all the 2+2 write tests we performed did not include any fences on the CPU or the GPU thread. Instead, we generated our different test configurations using different window sizes and number of observer threads. The first column in table A.7 shows the number of CPU observer threads, the second column shows the number of GPU observer threads, and the third column shows the

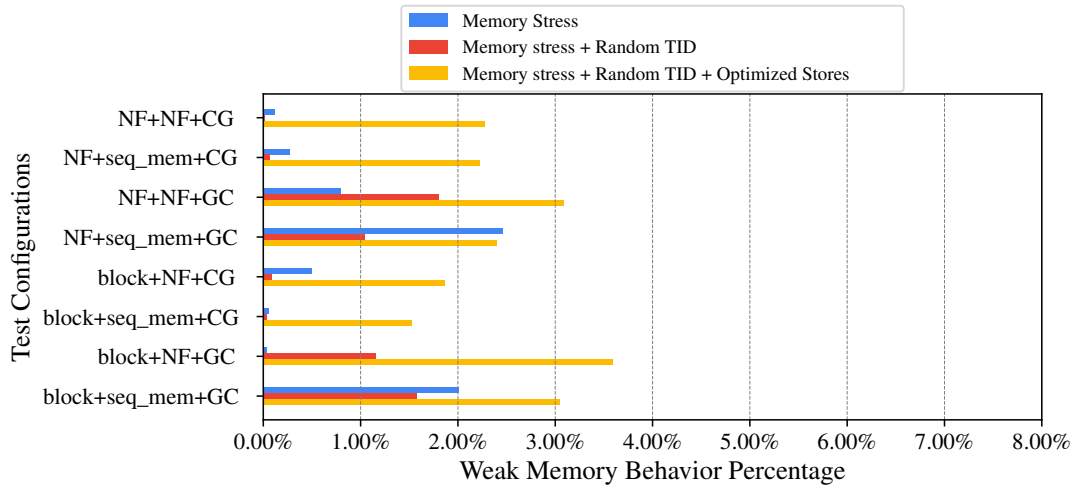


Figure 4.6: Weak memory behavior percentage observed with different optimizations for Store Buffer single instance test configurations.

window size. The fourth and fifth columns show whether stress was used and the direction of the test. Then the last column shows the throughput for weak memory behavior.

4.4.1 Analysis

- For perpetual instance tests, we observed that different litmus tests react differently to memory stress. In the single instance tests, we got a higher percent of weak behavior with all the litmus tests using the memory stress optimization. In fact, for the single instance tests, we rarely saw any weak behaviors without memory stress because of which we did not run our test suite without memory stress.

With the perpetual instance testing approach, the 2+2 Write litmus tests, Store Buffer tests, and Message Passing tests give a significantly higher weak memory throughput with memory stress. However, the Load Buffer tests perform better

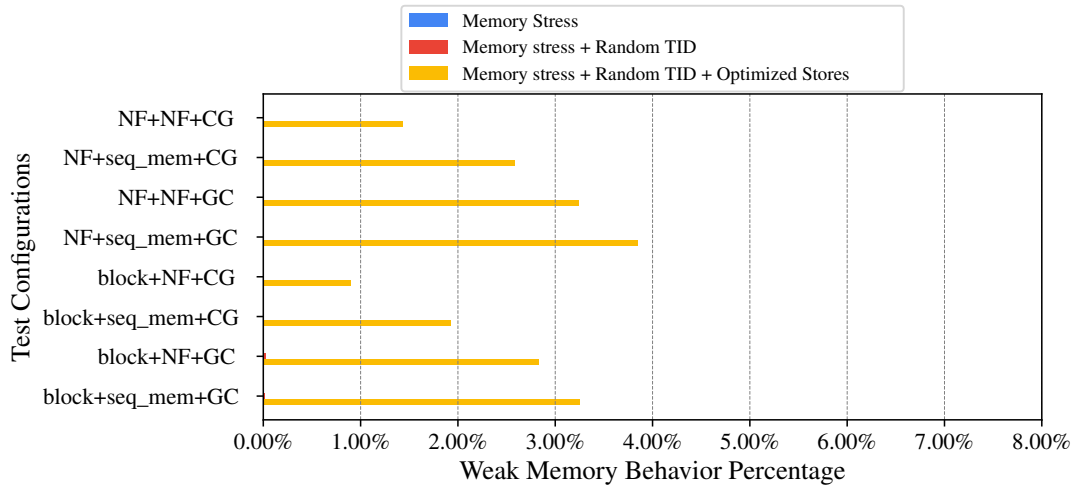


Figure 4.7: Weak memory behavior percentage observed with different optimizations for Store single instance test configurations.

without memory stress. Even though we could not record enough significant data for the Store test, Table 4.3 shows that from the 4 configurations in which we are able to see throughput higher than 0, 3 had no memory stress, and the last configuration with memory stress had significantly lower throughput than the others.

- Analysis of the graph in Figure 4.17 reveals that we get a higher throughput for a larger window size. Although we did not go beyond a window size of 5 in our experiments, our hypothesis was that the change in throughput with increasing window size would follow a bell curve. This is because, at some point, the overhead associated with increasing the window size should surpass the frequency of weak behaviors observed, giving us a lower throughput. To confirm our hypothesis, we did a small set of experiments where we tested the throughput variation with increasing window sizes for the 2+2 Write test with no

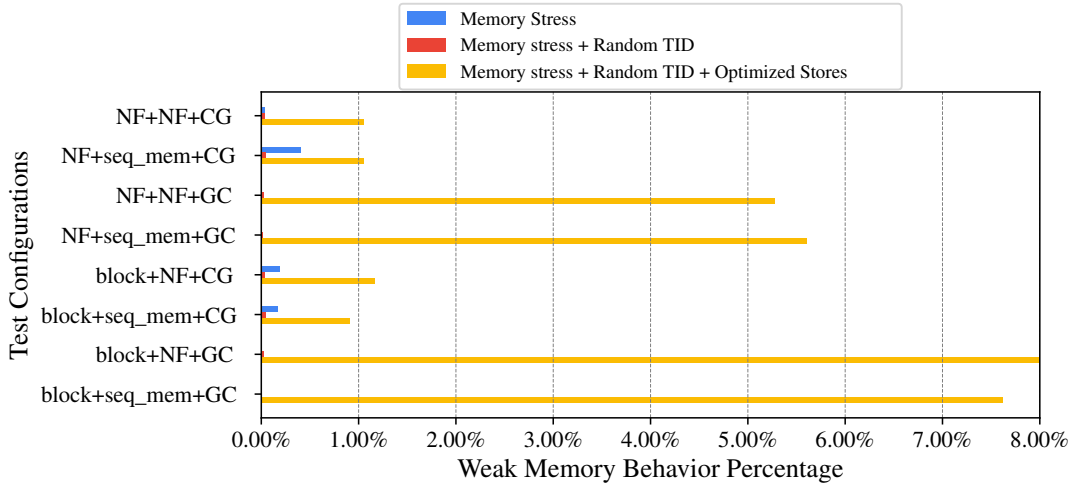


Figure 4.8: Weak memory behavior percentage observed with different optimizations for Read single instance test configurations.

fences and 1 CPU and GPU observer thread ran with 10,000 iterations. Figure 4.15 shows the throughput we observed with increasing window size, and we get a bell curve as expected. It is important to note that we might not get the peak at the same window size in the bell curve for different runs, even if the test configuration and the fuzzed parameters are the same. This is because the amount of weak behaviors we see, hence the shape and peak of the curve, depends upon the number of unique values recorded by the observer threads. If the observer threads stop recording unique values after an iteration t , then we should start seeing the decline in weak behavior throughput around iteration t .

- An interesting observation we made was that higher number of observer threads does not necessarily imply that we will record more weak behaviors. Previously, our hypothesis was that more observer buffers would help us in recording more events, through which we can infer more cycles. However, we get the best results

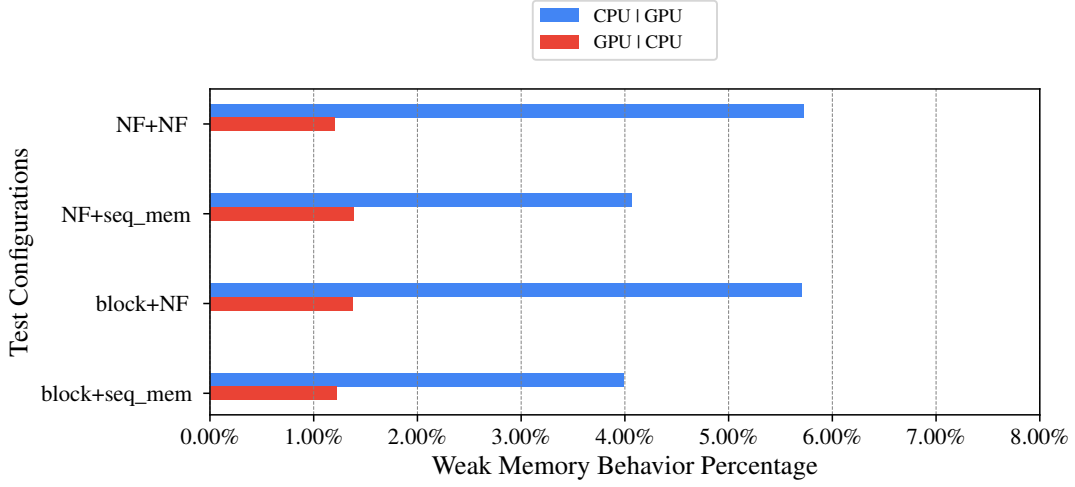


Figure 4.9: Test direction impact on weak behavior percentage for 2 + 2 Write single instance test configurations.

when we use 1 observer thread on the CPU and 1 on the GPU. The throughput even slightly decreases when we increase the number of observer threads.

- For perpetual instance tests, a million iterations with the single for loop heuristics take us an average of 2 seconds. Single-instance tests take an average of 9 seconds for 300 iterations. Therefore, perpetual instance tests give us about 15,000x speedup. This speedup is lower when we have the $\theta(n^2)$ complexity heuristics as in 2+2 write. But, we aim to achieve the same speedup with the 2+2 write test using the synchrony-point optimization which will reduce the 2 for loops to a single for loop. We are currently in the process of refining this optimization for the 2+2 Write tests and relaxing some conditions in it so we can record weak behaviors.
- We get an incredibly high throughput with the perpetual instance tests, as can be observed from the graphs. For single instance tests, only 3 tests (with all three

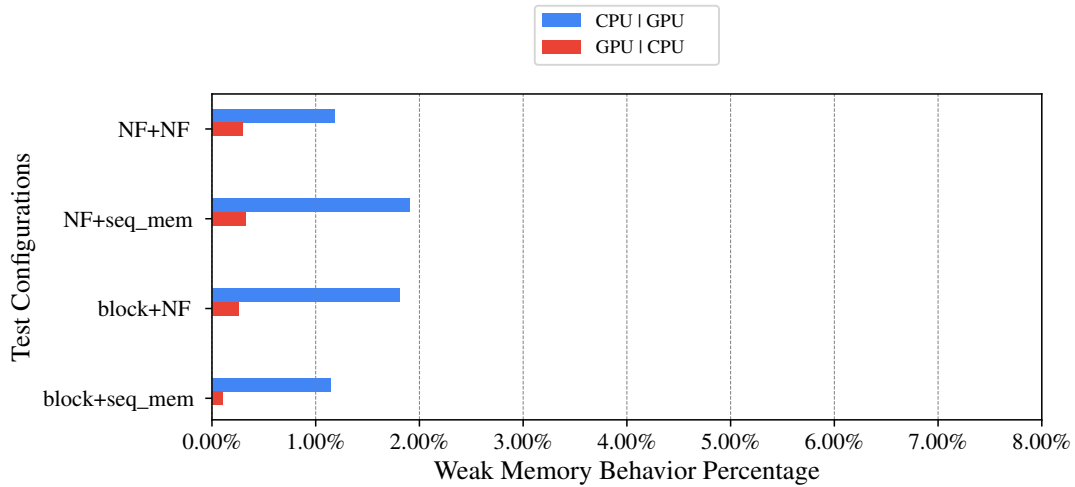


Figure 4.10: Test direction impact on weak behavior percentage for Load Buffer single instance test configurations.

optimizations added) had a higher throughput than 2 weak behaviors per second:

- Read litmus test with `blockfence` on the GPU thread; direction = GPU | CPU: Throughput = 2.66
- 2 + 2 Write litmus test with no fences on the CPU or GPU; direction = CPU | GPU: Throughput = 2.22
- Read litmus test with a CPU fence and a `blockfence` on the GPU thread; direction = GPU | CPU: Throughput = 2.52

The highest throughput for perpetual instance tests was 81,194.58 weak behaviors per second. This throughput came from the Message Passing litmus test with a CPU fence and a `blockfence` on the GPU thread, executed with memory stress enabled and in the reverse direction (thread 0 on the GPU and thread 1 on the CPU).

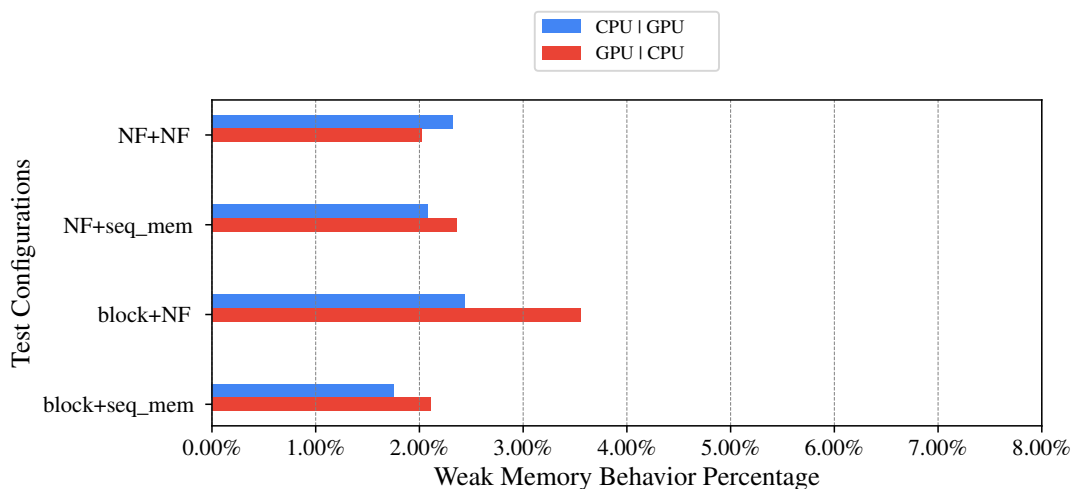


Figure 4.11: Test direction impact on weak behavior percentage for Message Passing single instance test configurations.

- Perpetual instance tests are not only efficient in giving us a significantly higher throughput, but also exposing behavior that the single instance tests were unable to. In the single instance tests, we were not able to observe any weak behaviors with a `threadfence` between instructions on the GPU thread. However, with the perpetual instance approach, we are able to observe weak memory behavior with a `threadfence` in the Message Passing and Store Buffer litmus tests. The highest throughput we observed with a `threadfence` on the GPU thread was 261 weak behaviors per second using the Message Passing test. These observations imply that a `threadfence` is not sufficient to eliminate all reorderings and weak behaviors. As described in section 4.3.1, a `threadfence` does not guarantee that the write updates will be propagated to the CPU. However, a `systemfence` does propagate the write updates to the CPU and the GPU, and for this reason, we see no weak behaviors when we use this fence in our perpetual tests. Hence, our

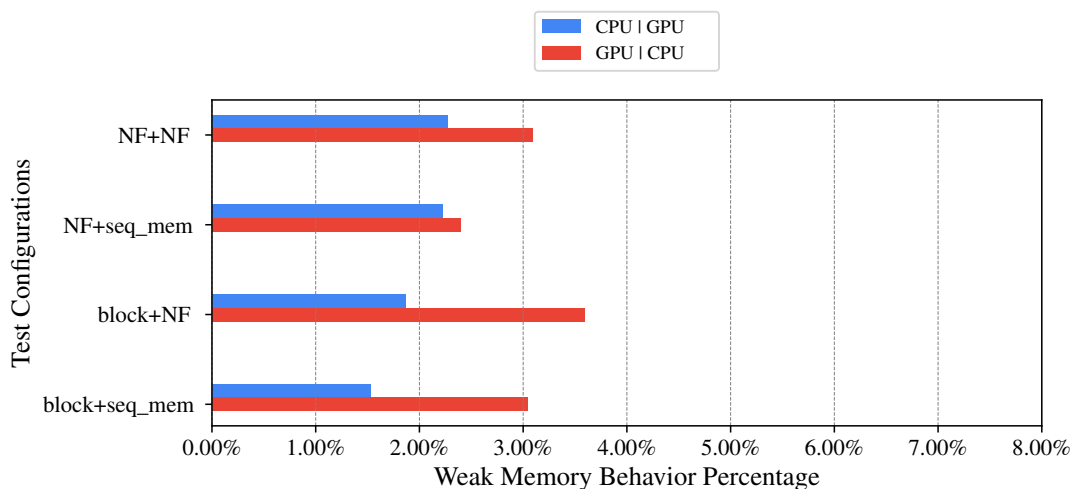


Figure 4.12: Test direction impact on weak behavior percentage for Store Buffer single instance test configurations.

observations are consistent with the documentation of these fences. To eliminate weak behaviors, we would have to use the strongest fence on the GPU, which is a `__threadfence_system()`.

- We see all four reorderings with perpetual instance tests, as with the case of single instance tests. The average weak behavior throughput of the Store litmus tests and the 2+2 write litmus tests is lesser than the others, but this is not because these reorderings don't happen on the system, but because our observer threads fail to record consistent unique data, giving us a lower throughput. Hence, it is possible that the reorderings happened, and we were just not able to record the values that corresponded to them through our observer threads.

The throughput for LB, SB, and MP tests is much higher because using the synchrony point optimization for them eliminates the need to use observer buffers in the trace analysis heuristics. If we compare the average throughput of configu-

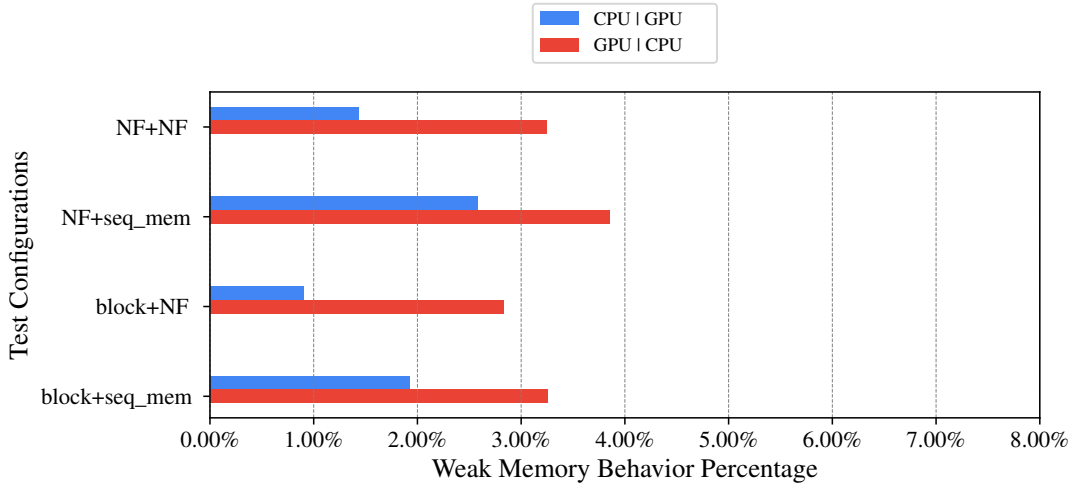


Figure 4.13: Test direction impact on weak behavior percentage for Store single instance test configurations.

rations of these three tests, then Load Buffer gives us the lowest throughput, and Message Passing gives the highest throughput. In our single instance test result analysis, we had inferred that the Load Buffer test gives us the least weak behavior percentage which is also our observation with the perpetual instance results. This solidifies our analysis that the Load $\rightarrow$ Store reordering is the least common on the system.

As in the case of single instance results, we see some tests giving a higher throughput in one of the directions. From graphs in Figures 4.18, 4.19, and 4.20, we can observe that the Load Buffer and Store Buffer litmus tests give a higher throughput in the CPU - GPU direction, and the Message Passing litmus test gives a higher throughput in the GPU - CPU direction. The Message Passing test is not symmetrical as it has Store $\rightarrow$ Store sequence on one thread and Load $\rightarrow$ Load sequence on another. We get better results when the Store $\rightarrow$ Store se-

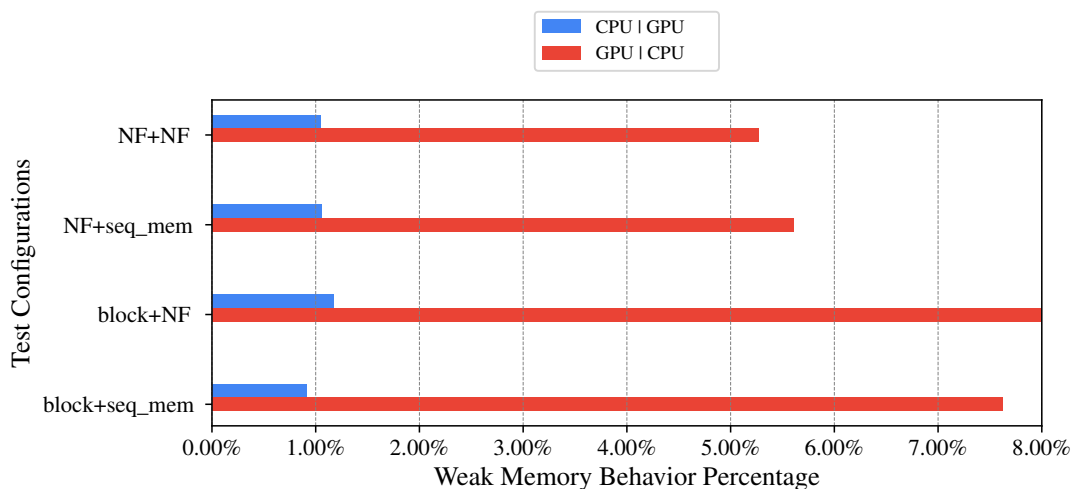


Figure 4.14: Test direction impact on weak behavior percentage for Read single instance test configurations.

sequence is executed on the GPU and the Load $\rightarrow$ Load sequence is executed on the CPU. This behavior could indicate that the GPU is more prone to the Store $\rightarrow$ Store reordering than the CPU, and the CPU is more prone to the Load $\rightarrow$ Load reordering than the GPU. It is interesting to see the weak behaviors skewed to one of the directions in the case of symmetrical tests. We currently hypothesize that this could be happening due to a number of internal system factors, as mentioned in the single instance test results analysis, and we plan to investigate the reasoning behind this behavior as part of future work. However, we strongly believe that this variation in the number of weak behaviors is not a result of false positives. This is because, firstly, we have confirmed that our tests and heuristics work on CPU systems and give us the expected results. If there were false positives, then we would have observed weak behaviors even after using fences on both CPU threads. However, all weak behaviors get eliminated when

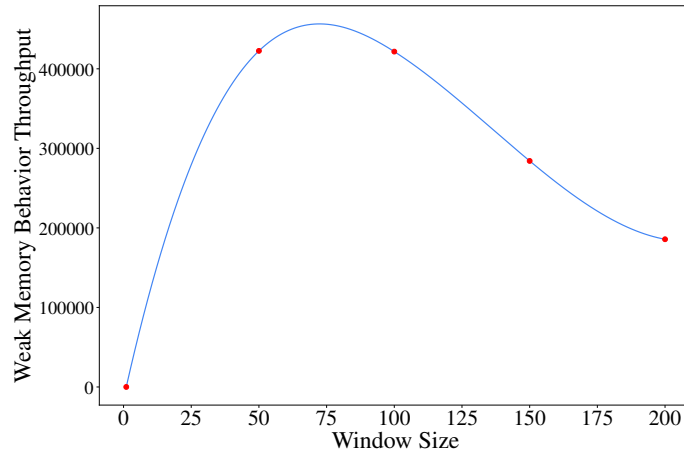


Figure 4.15: Change in throughput with increasing window size.

we have fences in between instructions on both CPU threads. Secondly, our tests are wrapped inside functions, which means each CPU and GPU test thread has its own function, and all the memory addresses and buffers are passed as function arguments. Because of this, the compiler should not be able to assume that the function arguments refer to different memory locations and reorder them.

- In our experiments, we always placed the fences in between the two instructions on the threads. We did not run our test suites with fences placed at the end or at the beginning of both instructions. However, we discuss here the behavior we expect if the fence placement were different in the threads.

– **If fences had been placed at the end of both instructions in the threads:**

Placing the CPU/GPU fence at the end of both instructions in the CPU and GPU threads would not have prevented any reorderings regardless of the strength of the GPU fences used. The reasoning behind this behavior is that a fence ensures that the instruction prior to it does not get reordered with the instruction following it. If both the test instructions were before the fence,

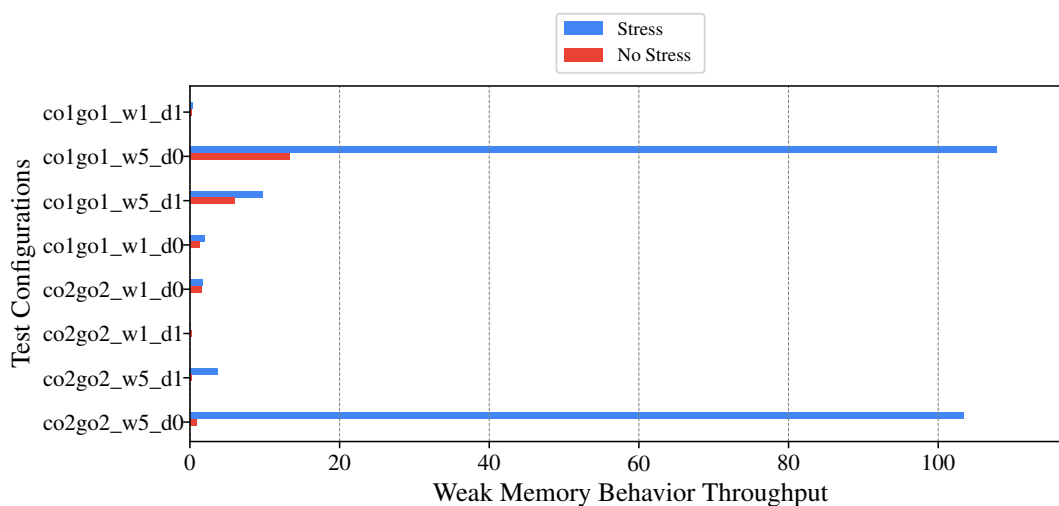


Figure 4.16: Throughput for different perpetual instance test configurations of 2+2 Write litmus test with and without stress.

then they could still be reordered. Hence, with this placement, we would have still seen weak behaviors.

To confirm our hypothesis, we ran our perpetual instance Message Passing litmus test (GPU CPU direction with memory stressing enabled) with two configurations. In the first configuration, we placed a `blockfence` at the end of both instructions in the GPU thread and a CPU fence after both instructions in the CPU thread. In the second configuration, we placed a `systemfence` at the end of both instructions in the GPU thread and a CPU fence at the end of both instructions in the CPU thread. We conducted several runs with these two configurations, and we were able to see a significant weak behavior throughput confirming our hypothesis.

- **If fences were placed before both instructions in the test threads** - This fence placement would not have prevented any reorderings. If both the

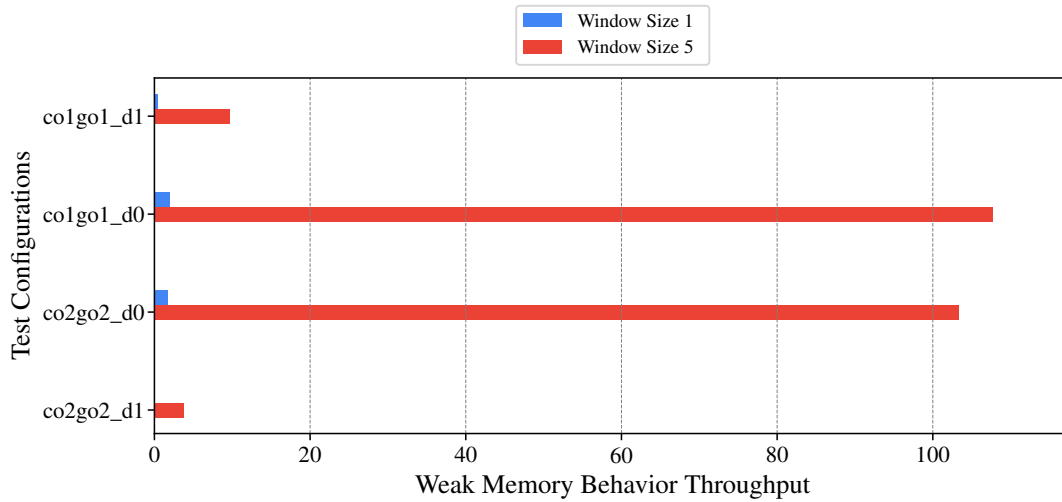


Figure 4.17: Impact of window size on weak memory behavior throughput for different perpetual instance test configurations of 2+2 Write litmus test with memory stress enabled.

fences were placed before the instructions in the CPU and GPU thread, then regardless of the strength of the CPU/GPU fence, it would not have ensured that the two instructions (which are after the fence) were not getting reordered. We can only restrict the reordering by placing the fences between the two-instruction sequence.

To confirm this speculation, we ran our perpetual instance Message Passing litmus test (GPU | CPU direction with memory stressing enabled) with two configurations. In one configuration, we placed a `blockfence` on top of both GPU thread instructions and a CPU fence on top of the CPU thread instructions. In the other configuration, we placed a `systemfence` on top of both instructions in the GPU thread and a CPU fence on top of both instructions in the CPU thread. We ran these two configurations multiple

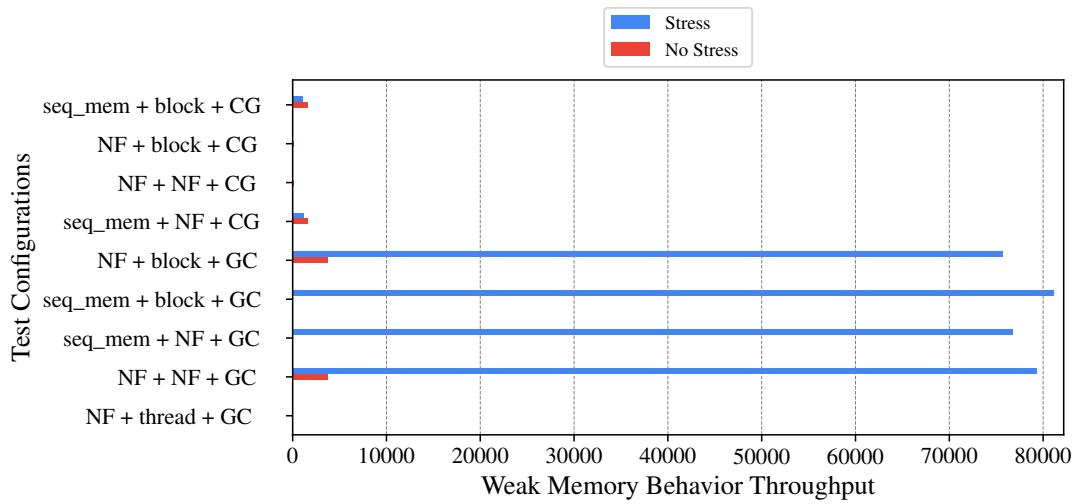


Figure 4.18: Weak memory behavior throughput for different test configurations of the MP litmus test using the perpetual litmus testing approach.

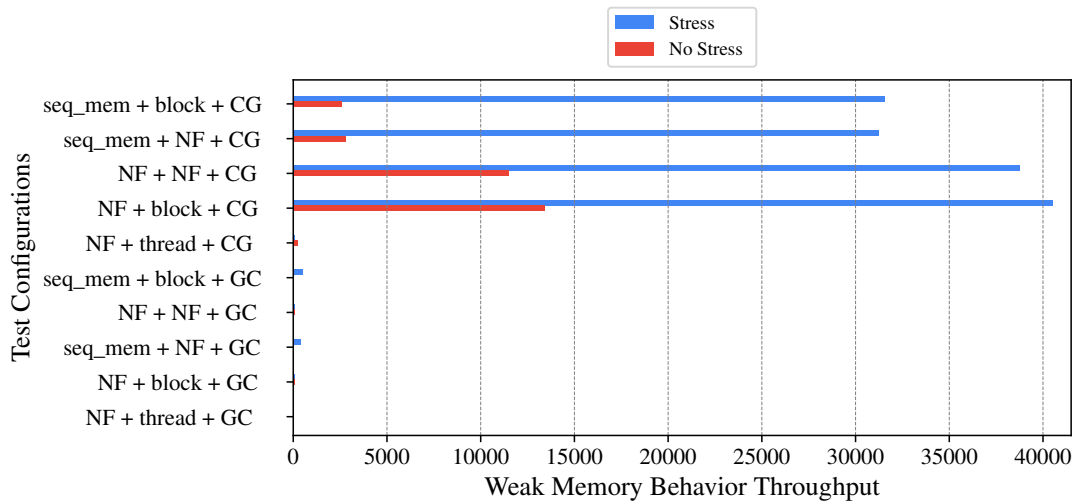


Figure 4.19: Weak memory behavior throughput for different test configurations of the SB litmus test using the perpetual litmus testing approach.

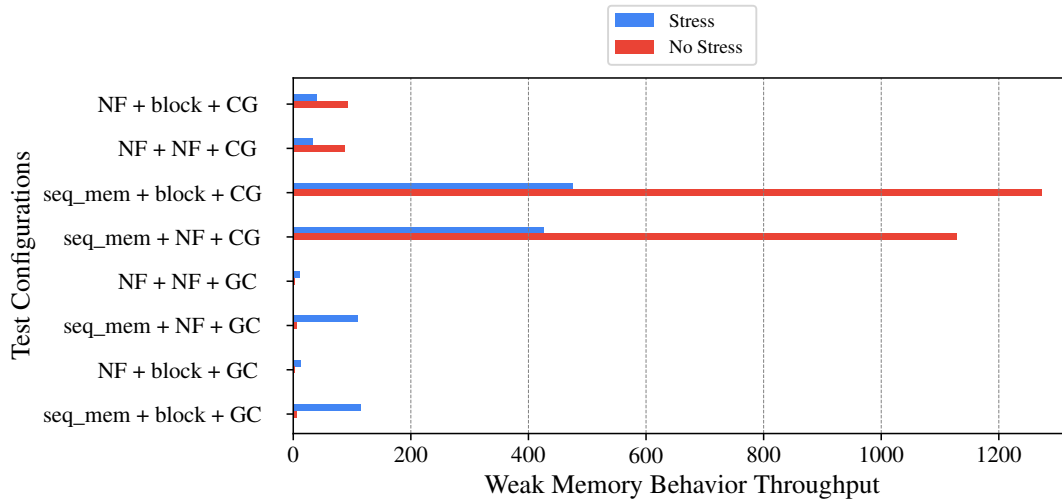


Figure 4.20: Weak memory behavior throughput for different test configurations of the LB litmus test using the perpetual litmus testing approach.

times, and we were still able to observe weak behaviors, which confirmed our initial analysis.

- **If a fence was placed between both instructions and another fence was placed at the end of the second instruction in the test threads:** The effect of this placement would be similar to placing a fence just between the two instructions, and the last fence after the two instructions would be redundant. Hence, the behavior of the test would depend upon the behavior of the fence used in between the two instructions and not on the fence used after the second instruction. On the CPU, we have the choice of using only one fence (`atomic_thread_fence(memory_order_seq_cst)`). However, we have different choices of fences to choose from on the GPU.

- * If we had a `blockfence` in between the two instructions in the GPU thread, followed by a `systemfence` after the second GPU thread in-

struction, and a CPU fence after the first and second instructions in the CPU thread, we would still have been able to see weak behaviors. This is because the fence in between the instructions decides whether the two-instruction sequence will be reordered or not. As we have shown in our experiments, a `blockfence` between the instructions is not enough to eliminate weak behaviors. Hence, in this configuration, we would have still observed weak behaviors.

- * If we had a `systemfence` after the first instruction in the GPU thread and a `blockfence` after the second instruction in the GPU thread with a CPU fence after the first and second instructions in the CPU thread, then all weak behaviors would have been eliminated. This is because a `systemfence` ensures that the two-instruction sequence would not get reordered. The fences at the end of both instructions are redundant.
- * If we had a `blockfence` after the first instruction in the GPU thread and a `blockfence` after the second instruction in the GPU thread with a CPU fence after the first and second instructions in the CPU thread, then we would still be able to see weak behaviors. This is because the fence between the GPU thread instructions does not guarantee that the write updates get propagated to the CPU.

To confirm all our analyses, we did similar experiments as in the previous cases using our perpetual instance Message Passing litmus test. We were able to see weak behaviors in all configurations except the ones in which we had a `systemfence` in between both instructions in the GPU thread. Although it is rare to observe weak behaviors using a `threadfence`, we know

from our experiments that weak behaviors can still happen using this fence. Hence, we expect to see weak behaviors if a `threadfence` is used in the GPU threads before both instructions, after both instructions, or in between both instructions. The only way by which we can eliminate reorderings is if we use a `systemfence` between both instructions in the GPU thread.

Chapter 5

Future Work and Conclusion

5.1 Future Work

- We have only focused on a small set of litmus tests. We aim to use our compiler for generating different kinds of tests, such as IRIW and IRRWIW (that have more than 2 threads) and coherence tests.
- We have emphasized before that our compiler can be extended to target other heterogeneous platforms. In the future, we plan to add other backends and test multiple devices.
- As described earlier in the thesis, we faced a number of issues with getting shared memory and CPU-GPU communication working on AMD and Intel devices. In the future, we plan to thoroughly investigate these devices and explore why memory stops getting updated after a certain period. We then plan on getting the results on these devices and formulating an analysis of how shared memory behaves and the strength of fences required to eliminate weak behaviors.

- We have ideas that are still in progress. As we have not formally analyzed them and their correctness, we refrained from introducing them in the thesis. We are currently exploring two such hypotheses/methodologies

– **Probabilistically estimating the percentage of weak behaviors:** For example, in the 2 + 2 write test, a weak memory behavior is the result of a *po* reordering between two store instructions. Using our observer buffers, we can infer whether there were any *po* reorderings, which can give us a probabilistic estimate of the number of weak behaviors we would have observed. We use the term ‘probabilistic’ because to confirm a weak behavior, we would have to check the entire cycle. However, if we observe a single *po* reordering, then it is proof that a Store $\rightarrow$ Store reordering happened and is possible, and there is a probability of observing weak behaviors with this test. As part of future work, we intend to analyze this hypothesis for its correctness and use it to reduce the time complexity of our heuristics.

– **Transforming 2+2 Write litmus test into the Message Passing test for observing reorderings:** We can use observer threads to convert a 2+2 Write test to a Message Passing litmus test. The Message Passing litmus test has a Store $\rightarrow$ Store sequence on one thread and a Load $\rightarrow$ Load sequence on the other. In the Store $\rightarrow$ Store sequence, we first store to X and then to another memory location, Y . In the Load $\rightarrow$ Load sequence, this order is reversed, and we first load from Y and then from X . Hence, to convert to a Message Passing test, we can use one of the threads from the 2+2 Write test for the Store $\rightarrow$ Store sequence and the observer thread for the Load $\rightarrow$ Load sequence. We might have to reverse the order of instructions in the

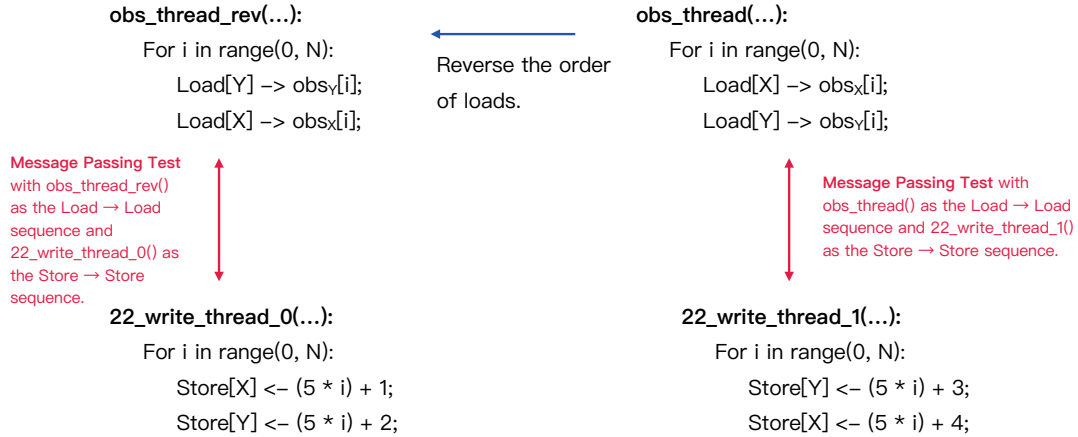


Figure 5.1: Converting the 2+2 Write litmus test into a Message Passing litmus test using observer threads.

observer thread depending on which thread we select from the 2+2 Write test. Figure 5.1 shows how we plan to do this conversion. When the 2 + 2 Write test thread is on the CPU, then the observer thread can be on the GPU. Similarly, when the 2 + 2 write test thread is on the GPU, then the observer thread can be on the CPU. When doing this conversion, we will not be removing the other 2 + 2 write test thread because it can behave as a stress thread. If we place a fence between both the load instructions in the observer thread, and still see weak behaviors, then the weak behaviors should come from Store → Store reordering (reordering in the 2 + 2 Write thread). We can use analyses and experiments like these to confirm whether observing a Store → Store reordering is possible. This analysis can help us infer whether it is possible to see a weak memory behavior using the 2 + 2 Write test because, as explained in the previous bullet point, weak behavior in the 2 + 2 Write litmus test is a result of the Store → Store reordering.

- We mentioned previously how our observer threads do not guarantee recording unique values. Because of this, our synchrony point optimization is not able to give us the expected outcome. The heuristics for our synchrony point optimization rely on how interleaved the observations are in different observer buffers. This interleaved behavior means that the observer thread should not be consistently recording values that come from a single node or iteration. We are currently working on resolving this, and we aim to explore tactics to make our observer threads more efficient in observing unique values.
- We plan to run our tests with different compiler optimizations to see if we can see any anomalies or different patterns. Our current hypothesis is that different compiler optimizations should not cause any effect on the weak behavior pattern we are currently observing.
- Through our results, we showed that memory stress is really efficient in giving us a higher percentage of weak behaviors in single instance tests. For the perpetual instance approach, some tests give us a higher throughput with memory stress, and some don't, as highlighted in Chapter 4. In our experiments, we used the same stress functions described in Section 3.6 in every run. We believe that different stress functions/patterns could cause different amounts of weak behaviors. We plan to explore and implement different stress patterns/functions to analyze if there are patterns that are most efficient in creating memory stress and giving a higher number of weak behaviors.

5.2 Conclusion

In conclusion, we built a compiler that serves as a tool for testing memory consistency models of heterogeneous systems. Our compiler is also the only compiler that takes a cyclic execution graph instead of the pseudo-code for the litmus tests. Additionally, we proposed the observer thread methodology to extend the perpetual instance testing approach and the synchrony point optimization to reduce the time complexity of the trace-analysis heuristics.

We obtained promising results with CUDA using both the single instance and perpetual instance approach on the 8GB NVIDIA Jetson Orin Nano. We showed that the perpetual testing approach gives us a significant speedup and higher throughput than single instance tests when using single for-loop trace analysis heuristics. Notably, we observed all four reorderings on this device. To restrict the weak memory behaviors, we determined that a `systemfence` would have to be used in between the instructions in the GPU thread, and using a `threadfence` or fences on the CPU alone might not be enough to eliminate these reorderings.

Overall, this is an ongoing project, and several components require further development. We plan to confirm our analysis hypotheses by investigating the architecture and understanding how the communication between the CPU and GPU happens. This investigation would provide more insights into the trends we highlighted in Chapter 4. With this, we also plan to make our compiler more versatile by adding different backends, especially so we can test other systems, such as Apple devices.

Appendix A

Raw Data

The Tables [A.1](#), [A.2](#) [A.3](#), [A.4](#), [A.5](#), and [A.6](#) show the results we obtained on the NVIDIA Jetson Orin Nano using the single instance approach. The Tables [A.7](#), [A.8](#), [A.9](#), and [A.10](#) show the results we obtained from our perpetual instance test experiments on the NVIDIA Jetson Orin Nano.

Direction	GPU Fence	CPU Fence	Memory Stress	Memory Stress + Random TID	Memory Stress + Random TID + Optimized Stores
CPU GPU	NF	NF	0.54700%	0.5627%	2.3156%
CPU GPU	NF	seq_mem	0.67300%	3.8326%	2.0794%
GPU CPU	NF	NF	0.00000%	0.0148%	2.0220%
GPU CPU	NF	seq_mem	0.00000%	0.0511%	2.3532%
GPU CPU	block	NF	0.00000%	0.0037%	3.5504%
CPU GPU	block	seq_mem	0.47470%	3.9836%	1.7546%
CPU GPU	block	NF	0.00492%	4.7252%	2.4390%
GPU CPU	block	seq_mem	0.00000%	0.0209%	2.1092%
GPU CPU	system	NF	0.00000%	0.0000%	0.0000%
CPU GPU	system	seq_mem	0.00000%	0.0000%	0.0000%
CPU GPU	system	NF	0.00000%	0.0000%	0.0000%
GPU CPU	system	seq_mem	0.00000%	0.0000%	0.0000%
CPU GPU	threadfence	NF	-	-	0.0000%
GPU CPU	threadfence	NF	-	-	0.0000%
CPU GPU	threadfence	seq_mem	-	-	0.0000%
GPU CPU	threadfence	seq_mem	-	-	0.0000%

Table A.1: Message Passing litmus test - single instance results.

Direction	GPU Fence	CPU Fence	Memory Stress	Memory Stress + Random TID	Memory Stress + Random TID + Optimized Stores
CPU GPU	NF	NF	0.1150%	0.0149%	2.2733%
CPU GPU	NF	seq_mem	0.2676%	0.0659%	2.2207%
GPU CPU	NF	NF	0.7990%	1.8030%	3.0874%
GPU CPU	NF	seq_mem	2.4575%	1.0474%	2.4000%
CPU GPU	block	NF	0.4960%	0.0908%	1.8615%
CPU GPU	block	seq_mem	0.0549%	0.0385%	1.5304%
GPU CPU	block	NF	0.0317%	1.1548%	3.5881%
GPU CPU	block	seq_mem	2.0121%	1.5770%	3.0437%
CPU GPU	system	NF	0.0000%	0.0000%	0.0000%
CPU GPU	system	seq_mem	0.0000%	0.0000%	0.0000%
GPU CPU	system	NF	0.0000%	0.0000%	0.0000%
GPU CPU	system	seq_mem	0.0000%	0.0000%	0.0000%
CPU GPU	threadfence	NF	-	-	0.0000%
GPU CPU	threadfence	NF	-	-	0.0000%
CPU GPU	threadfence	seq_mem	-	-	0.0000%
GPU CPU	threadfence	seq_mem	-	-	0.0000%

Table A.2: Store Buffer litmus test - single instance results.

Direction	GPU Fence	CPU Fence	Memory Stress	Memory Stress + Random TID	Memory Stress + Random TID + Optimized Stores
CPU GPU	NF	NF	0.0000%	0.0000%	1.1852%
CPU GPU	NF	seq_mem	0.0000%	0.0000%	1.9037%
GPU CPU	NF	NF	0.0000%	0.0000%	0.2919%
GPU CPU	NF	seq_mem	0.0000%	0.0000%	0.3215%
CPU GPU	block	NF	0.0000%	0.0000%	1.8074%
CPU GPU	block	seq_mem	0.0000%	0.0000%	1.1467%
GPU CPU	block	NF	0.0000%	0.0000%	0.2519%
GPU CPU	block	seq_mem	0.0000%	0.0000%	0.1052%
CPU GPU	system	NF	0.0000%	0.0000%	0.0000%
CPU GPU	system	seq_mem	0.0000%	0.0000%	0.0000%
GPU CPU	system	NF	0.0000%	0.0000%	0.0000%
GPU CPU	system	seq_mem	0.0000%	0.0000%	0.0000%
CPU GPU	threadfence	NF	-	-	0.0000%
GPU CPU	threadfence	NF	-	-	0.0000%
CPU GPU	threadfence	seq_mem	-	-	0.0000%
GPU CPU	threadfence	seq_mem	-	-	0.0000%

Table A.3: Load Buffer litmus test - single instance results.

Direction	GPU Fence	CPU Fence	Memory Stress	Memory Stress + Random TID	Memory Stress + Random TID + Optimized Stores
CPU GPU	NF	NF	0.0327%	0.0393%	1.0489%
CPU GPU	NF	seq_mem	0.4010%	0.0481%	1.0533%
GPU CPU	NF	NF	0.0000%	0.0252%	5.2733%
GPU CPU	NF	seq_mem	0.0000%	0.0148%	5.6022%
CPU GPU	block	NF	0.1940%	0.0370%	1.1681%
CPU GPU	block	seq_mem	0.1700%	0.0511%	0.9096%
GPU CPU	block	NF	0.0000%	0.0215%	8.0370%
GPU CPU	block	seq_mem	0.0000%	0.0022%	7.6170%
CPU GPU	system	NF	0.0000%	0.0000%	0.0000%
CPU GPU	system	seq_mem	0.0000%	0.0000%	0.0000%
GPU CPU	system	NF	0.0000%	0.0000%	0.0000%
GPU CPU	system	seq_mem	0.0000%	0.0000%	0.0000%
CPU GPU	threadfence	NF	-	-	0.0000%
GPU CPU	threadfence	NF	-	-	0.0000%
CPU GPU	threadfence	seq_mem	-	-	0.0000%
GPU CPU	threadfence	seq_mem	-	-	0.0000%

Table A.4: Read litmus test - single instance results.

Direction	GPU Fence	CPU Fence	Memory Stress	Memory Stress + Random TID	Memory Stress + Random TID + Optimized Stores
CPU GPU	NF	NF	0.0000%	0.0000%	1.4300%
CPU GPU	NF	seq_mem	0.0000%	0.0000%	2.5800%
GPU CPU	NF	NF	0.0000%	0.0030%	3.2400%
GPU CPU	NF	seq_mem	0.0000%	0.0000%	3.8500%
CPU GPU	block	NF	0.0000%	0.0000%	0.8990%
CPU GPU	block	seq_mem	0.0000%	0.0000%	1.9235%
GPU CPU	block	NF	0.0000%	0.0296%	2.8278%
GPU CPU	block	seq_mem	0.0000%	0.0156%	3.2554%
CPU GPU	system	NF	0.0000%	0.0000%	0.0000%
CPU GPU	system	seq_mem	0.0000%	0.0000%	0.0000%
GPU CPU	system	NF	0.0000%	0.0000%	0.0000%
GPU CPU	system	seq_mem	0.0000%	0.0000%	0.0000%
CPU GPU	threadfence	NF	-	-	0.0000%
GPU CPU	threadfence	NF	-	-	0.0000%
CPU GPU	threadfence	seq_mem	-	-	0.0000%
GPU CPU	threadfence	seq_mem	-	-	0.0000%

Table A.5: Store litmus test - single instance results.

Direction	GPU Fence	CPU Fence	Memory Stress	Memory Stress + Random TID	Memory Stress + Random TID + Optimized Stores
CPU GPU	NF	NF	0.0279%	0.0393%	5.7200%
CPU GPU	NF	seq_mem	0.0144%	0.0407%	4.0622%
GPU CPU	NF	NF	0.0000%	0.0463%	1.2044%
GPU CPU	NF	seq_mem	0.0000%	0.0091%	1.3881%
CPU GPU	block	NF	0.0278%	0.0218%	5.7007%
CPU GPU	block	seq_mem	0.0211%	0.0554%	3.9867%
GPU CPU	block	NF	0.0000%	0.0119%	1.3778%
GPU CPU	block	seq_mem	0.0000%	0.0014%	1.2178%
CPU GPU	system	NF	0.0000%	0.0000%	0.0000%
CPU GPU	system	seq_mem	0.0000%	0.0000%	0.0000%
GPU CPU	system	NF	0.0000%	0.0000%	0.0000%
GPU CPU	system	seq_mem	0.0000%	0.0000%	0.0000%
CPU GPU	threadfence	NF	-	-	0.0000%
GPU CPU	threadfence	NF	-	-	0.0000%
CPU GPU	threadfence	seq_mem	-	-	0.0000%
GPU CPU	threadfence	seq_mem	-	-	0.0000%

Table A.6: 2 + 2 Write litmus test - single instance results.

CPU Observer Threads	GPU Observer Threads	Window Size	stress	Direction	Throughput
1	1	1	TRUE	GPU CPU	0.4005
1	1	5	TRUE	CPU GPU	107.7698
1	1	5	TRUE	GPU CPU	9.6755
1	1	1	TRUE	CPU GPU	2.0078
2	2	1	FALSE	CPU GPU	1.5024
1	1	1	FALSE	CPU GPU	1.2557
1	1	1	FALSE	GPU GPU	0.1955
2	2	1	FALSE	GPU CPU	0.1822
2	2	5	FALSE	GPU CPU	0.228
1	1	5	FALSE	GPU CPU	5.8975
1	1	5	FALSE	CPU GPU	13.2935
2	2	5	FALSE	CPU GPU	0.8821
2	2	1	TRUE	CPU GPU	1.7318
2	2	5	TRUE	GPU CPU	3.7435
2	2	5	TRUE	CPU GPU	103.3791
2	2	1	TRUE	GPU CPU	0.0628

Table A.7: 2 + 2 write litmus test - perpetual instance results.

CPU Fence	GPU Fence	Stress	Direction	Throughput
NF	block	TRUE	CPU GPU	40.6994
seq_mem	system	FALSE	CPU GPU	0
NF	system	FALSE	CPU GPU	0
NF	system	FALSE	GPU CPU	0
seq_mem	system	FALSE	GPU CPU	0
NF	block	TRUE	GPU CPU	12.5122
seq_mem	NF	FALSE	CPU GPU	1128.294
NF	NF	FALSE	CPU GPU	86.8031
NF	system	TRUE	CPU GPU	0
NF	system	TRUE	GPU CPU	0
NF	NF	FALSE	GPU CPU	2.4094
seq_mem	NF	FALSE	GPU CPU	6.0343
seq_mem	system	TRUE	CPU GPU	0
seq_mem	system	TRUE	GPU CPU	0
seq_mem	block	TRUE	CPU GPU	475.2495
seq_mem	block	TRUE	GPU CPU	114.3176
seq_mem	NF	TRUE	GPU CPU	110.4914
seq_mem	NF	TRUE	CPU GPU	426.7593
seq_mem	block	FALSE	GPU CPU	5.933
NF	block	FALSE	GPU CPU	1.979
NF	block	FALSE	CPU GPU	92.1604
seq_mem	block	FALSE	CPU GPU	1273.0018
NF	NF	TRUE	GPU CPU	11.8992
NF	NF	TRUE	CPU GPU	33.7552
NF	threadfence	TRUE	CPU GPU	0.000
NF	threadfence	FALSE	CPU GPU	0.000
NF	threadfence	TRUE	GPU CPU	0.000
NF	threadfence	FALSE	GPU CPU	0.000

Table A.8: Load Buffer litmus test - perpetual instance results.

CPU Fence	GPU Fence	Stress	Direction	Throughput
seq_mem	system	FALSE	CPU GPU	0
NF	system	FALSE	CPU GPU	0
NF	system	FALSE	GPU CPU	0
seq_mem	system	FALSE	GPU CPU	0
seq_mem	block	TRUE	GPU CPU	510.7255
seq_mem	block	TRUE	CPU GPU	31583.057
seq_mem	system	TRUE	GPU CPU	0
seq_mem	system	TRUE	CPU GPU	0
seq_mem	NF	FALSE	CPU GPU	2822.5745
NF	NF	FALSE	CPU GPU	11475.734
NF	NF	FALSE	GPU CPU	62.7763
seq_mem	NF	FALSE	GPU CPU	0
NF	system	TRUE	GPU CPU	0
NF	system	TRUE	CPU GPU	0
NF	block	TRUE	GPU CPU	74.5322
NF	block	TRUE	CPU GPU	40501.5719
NF	NF	TRUE	CPU GPU	38780.9405
NF	NF	TRUE	GPU CPU	66.6517
seq_mem	block	FALSE	GPU CPU	0.0041
NF	block	FALSE	GPU CPU	58.641
NF	block	FALSE	CPU GPU	13429.6779
seq_mem	block	FALSE	CPU GPU	2581.6127
seq_mem	NF	TRUE	CPU GPU	31256.1659
seq_mem	NF	TRUE	GPU CPU	399.1962
NF	threadfence	TRUE	CPU GPU	73.1487
NF	threadfence	FALSE	CPU GPU	261.7241
NF	threadfence	TRUE	GPU CPU	0.000
NF	threadfence	FALSE	GPU CPU	0.0091

Table A.9: Store Buffer litmus test - perpetual instance results.

CPU Fence	GPU Fence	Stress	Direction	Throughput
seq_mem	system	TRUE	CPU GPU	0
seq_mem	block	FALSE	CPU GPU	1630.9263
NF	block	FALSE	CPU GPU	86.4699
NF	block	FALSE	GPU CPU	3734.4568
seq_mem	block	FALSE	GPU CPU	4.5497
seq_mem	system	TRUE	GPU CPU	0
seq_mem	block	TRUE	CPU GPU	1101.3092
seq_mem	block	TRUE	GPU CPU	81194.5879
NF	block	TRUE	CPU GPU	57.4378
NF	block	TRUE	GPU CPU	75759.6647
NF	system	TRUE	CPU GPU	0
NF	system	TRUE	GPU CPU	0
seq_mem	NF	FALSE	GPU CPU	3.7169
NF	NF	FALSE	GPU CPU	3803.7459
NF	NF	FALSE	CPU GPU	82.1172
seq_mem	NF	FALSE	CPU GPU	1648.996
seq_mem	system	FALSE	GPU CPU	0
NF	system	FALSE	GPU CPU	0
NF	system	FALSE	CPU GPU	0
seq_mem	system	FALSE	CPU GPU	0
NF	NF	TRUE	GPU CPU	79365.6122
NF	NF	TRUE	CPU GPU	56.6017
seq_mem	NF	TRUE	GPU CPU	76824.6068
seq_mem	NF	TRUE	CPU GPU	1141.679
NF	threadfence	TRUE	CPU GPU	0.000
NF	threadfence	FALSE	CPU GPU	0.000
NF	threadfence	TRUE	GPU CPU	0.000
NF	threadfence	FALSE	GPU CPU	4.6366

Table A.10: Message Passing litmus test - perpetual instance results.

Bibliography

- [1] Sarita V. Adve and Kourosh Gharachorloo. Shared memory consistency models: A tutorial. *Computer*, 29(12):66–76, dec 1996.
- [2] Jade Alglave. *A Shared Memory Poetics*. PhD thesis, 2010.
- [3] Jade Alglave. A formal hierarchy of weak memory models. *Form. Methods Syst. Des.*, 41(2):178–210, oct 2012.
- [4] Jade Alglave, Mark Batty, Alastair F. Donaldson, Ganesh Gopalakrishnan, Jeroen Ketema, Daniel Poetzl, Tyler Sorensen, and John Wickerson. Gpu concurrency: Weak behaviours and programming assumptions. *SIGPLAN Not.*, 50(4):577–591, mar 2015.
- [5] Andrés Goens, Soham Chakraborty, Susmit Sarkar, Sukarn Agarwal, Nicolai Oswald, and Vijay Nagarajan. Compound memory models. *Proc. ACM Program. Lang.*, 7(PLDI), jun 2023.
- [6] Christoph Hagleitner, Florian Auernhammer, James Sexton, Constantinos Evangelinos, Guernsey Hunt, Christian Pinto, Michael Johnston, Charles Johns, and Jim Kahle. An architecture for heterogeneous high-performance computing systems: Motivation and requirements. In *2023 IEEE John Vincent Atanasoff International Symposium on Modern Computing (JVA)*, pages 14–14, 2023.
- [7] HiSilicon. Kirin 9000. <https://www.hisilicon.com/en/products/Kirin/Kirin-flagship-chips/Kirin-9000>. Accessed: 2024-06-05.
- [8] INRIA. Running tests with litmus7. <https://diy.inria.fr/doc/litmus.html>. Accessed: 2024-05-04.
- [9] Dan Iorga, Alastair F. Donaldson, Tyler Sorensen, and John Wickerson. The semantics of shared memory in intel cpu/fpga systems. *Proc. ACM Program. Lang.*, 5(OOPSLA), oct 2021.

- [10] Jake Kirkham, Tyler Sorensen, Esin Tureci, and Margaret Martonosi. Foundations of empirical memory consistency testing. *Proc. ACM Program. Lang.*, 4(OOP-SLA), nov 2020.
- [11] Lamport. How to make a multiprocessor computer that correctly executes multiprocess programs. *IEEE Transactions on Computers*, C-28(9):690–691, 1979.
- [12] Reese Levine, Mingun Cho, Devon McKee, Andrew Quinn, and Tyler Sorensen. Gpuharbor: Testing gpu memory consistency at large (experience paper). In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA 2023, page 779–791, New York, NY, USA, 2023. Association for Computing Machinery.
- [13] Reese Levine, Tianhao Guo, Mingun Cho, Alan Baker, Raph Levien, David Neto, Andrew Quinn, and Tyler Sorensen. Mc mutants: Evaluating and improving testing for memory consistency specifications. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ASPLOS 2023, page 473–488, New York, NY, USA, 2023. Association for Computing Machinery.
- [14] Luc Maranget, Susmit Sarkar, and Peter Sewell. A tutorial introduction to the arm and power relaxed memory models. *Draft available from <http://www.cl.cam.ac.uk/~pes20/ppc-supplemental/test7.pdf>*, 2012.
- [15] Themis Melissaris, Markos Markakis, Kelly Shaw, and Margaret Martonosi. Perple: Improving the speed and effectiveness of memory consistency testing. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 329–341, 2020.
- [16] Vijay Nagarajan, Daniel J. Sorin, Mark D. Hill, David A. Wood, and Natalie Enright Jerger. *A Primer on Memory Consistency and Cache Coherence*. Morgan & Claypool Publishers, 2nd edition, 2020.
- [17] NVIDIA. Cuda c++ programming guide. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html>. Accessed: 2024-05-04.
- [18] S. Sarkar, P. Sewell, J. Alglave, L. Maranget, and D. Williams. Understanding power multiprocessors. Accessed on: June 5, 2024.
- [19] Susmit Sarkar, Peter Sewell, Jade Alglave, Luc Maranget, and Derek Williams. Understanding power multiprocessors. *SIGPLAN Not.*, 46(6):175–186, jun 2011.

- [20] Snapdragon 888 5G Mobile Platform. <https://www.qualcomm.com/products/mobile/snapdragon/smartphones/snapdragon-8-series-mobile-platforms/snapdragon-888-5g-mobile-platform>. Accessed: 2024-06-05.
- [21] Tyler Sorensen and Alastair F. Donaldson. Exposing errors related to weak memory in gpu applications. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '16, page 100–113, New York, NY, USA, 2016. Association for Computing Machinery.
- [22] Wen-Mei W. Hwu. *Heterogeneous System Architecture: A new compute platform infrastructure*. Morgan Kaufmann, 2015.